BRACE: Unified Benchmarking of Accuracy and Energy for Code Language Models

MOHAMMADJAVAD MEHDITABAR, Dalhouise University, Canada
SAURABHSINGH RAJPUT, Dalhousie University, Canada
ANTONIO MASTROPAOLO, William & Mary, USA
TUSHAR SHARMA, Dalhouise University, Canada

The rapid adoption of AI in software development calls for evaluating the environmental cost of code models alongside their functional correctness. While prior studies have examined sustainability in large language models, they lack a systematic and interpretable framework for jointly evaluating the accuracy, energy, and size trade-offs of Code Language Models (*CLM*s). We present BRACE, a framework that benchmarks *CLM*s across energy efficiency, functional correctness, and model size, and reports them on a unified 1–5 scale. Rather than collapsing these competing dimensions into a single opaque score, BRACE decouples the evaluation into three interpretable pairwise profiles. For the energy–accuracy trade-off, we propose two rating methods. Concentric Incremental Rating Circles (CIRC) provides deterministic, distance-based rankings with static trade-offs that resist outliers, and Observation to Expectation Rating (OTER) provides trend-aware evaluation with dynamic trade-offs that capture complex correlation between energy and accuracy. We extend both methods with priority weighting so users can tune the relative importance of accuracy and energy. To account for deployment scale, we further introduce scaling-law-based performance and structural efficiency metrics that assess how effectively a model converts its size into accuracy and energy savings. Benchmarking 33 state-of-the-art *CLM*s on code generation and summarization, we find that models generally perform better on summarization, since it does not enforce syntactic correctness, and that small model size does not by itself secure efficiency. What separates models is how densely they convert parameters into capability, not raw scale. The resulting profiles act as portable, multi-dimensional tags spanning energy, accuracy, and size, helping practitioners select models that are accurate, sustainable, and deployable, and helping developers locate where a model wastes its parameters.

Additional Key Words and Phrases: Code Language Models, benchmarking, sustainability rating, energy consumption, green AI.

1 Introduction

Generative Artificial Intelligence (GenAI) tools powered by Large Language Models (LLMs) have achieved widespread adoption [15] due to their ability to generate human-like text, process complex queries, and assist users across various tasks [106]. In software engineering (SE), Large Language Models for Code (*CLM*s) such as Codex [23], which powers GitHub Copilot [38], are trained on vast corpora of source code and natural language to support development activities. These models generate, analyze, debug, and optimize code across multiple programming languages [31, 46, 53], and they are increasingly integrated into day-to-day workflows [70, 82, 113]. They also continue to evolve rapidly, narrowing capability gaps with humans while expanding to new SE tasks [46, 103]. This evolution stems primarily from scaling laws, that is, increasing model size to better capture the semantic space of SE tasks and thereby improve downstream accuracy [56]. However, this reliance on scale alone introduces three interlinked challenges. First, it carries severe environmental costs, because larger models consume drastically more energy [16, 75, 98] and emit more carbon [30, 59]. Second, it creates an operational bottleneck, because a model may be highly accurate while its memory footprint renders it infeasible for edge deployment or consumer-grade hardware. Third, it introduces a pervasive "Large Model

Authors' Contact Information: Mohammadjavad Mehditabar, Dalhouise University, Halifax, Canada, javad@dal.ca; Saurabhsingh Rajput, Dalhousie University, Halifax, Canada, saurabh@dal.ca; Antonio Mastropaolo, William & Mary, Williamsburg, USA, amastropaolo@wm.edu; Tushar Sharma, Dalhouise University, Halifax, Canada, tushar@dal.ca.

Bias" into evaluation ecosystems, because comparing a 70B parameter model directly against a 7B parameter model obscures architectural innovation behind raw physical scale. A massive model may achieve high accuracy yet suffer from extreme structural bloat, using its parameters far less efficiently than a smaller, denser model.

Efforts to benchmark and mitigate these challenges have grown, yet they rarely evaluate the three dimensions together. A new line of work targets the environmental sustainability of AI models alone. For example, AI Energy Score [64] benchmarks LLMs solely on their energy consumption, assigning a star-rating from 1 to 5. However, rating LLMs on sustainability metrics alone gives an incomplete picture that ignores functional correctness and model size, both of which are central to practical adoption, and excluding them can be a show-stopper in an accuracy-first or resource-constrained setting. Evaluating accuracy and energy while ignoring model size likewise disconnects the evaluation from deployment reality, because a model may hold a favorable energy-accuracy profile yet demand a memory allocation that rules it out on consumer-grade hardware.

Among the works that do pair accuracy with energy, a few studies [7, 48] apply Pareto optimality to identify non-dominated points, where no model outperforms another on both objectives. This line of work ignores model size as a trade-off criterion, and it faces three limitations in the accuracy–energy setting itself. First, it surfaces only the dominant points and leaves the rest uncharacterized. Second, without a function that combines the objectives, it cannot compare points on the frontier. Third, points on the frontier can differ widely in overall standing, because a model with an extreme value in one objective is neither rewarded nor penalized fairly. Beyond the frontier view, several approaches combine accuracy and energy into a single score. Kaplan *et al.* [55] combine *F1-score* and carbon emissions through the ratio of normalized carbon emission to F1-score, but a ratio assumes a linear relationship and overlooks the non-linear interaction between the two metrics. Masciari *et al.* [69] integrate carbon efficiency with functional correctness through a linear formula, which carries the same limitation of a pre-defined linear form that is assumed rather than validated. Jegham *et al.* [51] build a scoring scheme on Data Envelopment Analysis (DEA) [22] that measures how effectively each model converts sustainability-related inputs into accurate outputs, but the approach is prone to overfitting [41], sensitivity to outliers [20], and ranking instability [100].

A smaller body of work adds memory footprint alongside energy and accuracy, but it still does not model the trade-off fairly. Gjorgjevikj *et al.* [39] apply PROMETHEE II, whose linear preference aggregation cannot capture non-linear covariance among the three dimensions. Fischer *et al.* [33] discretize each metric independently and then merge them through a weighted median. This early discretization erases the continuous trade-off dynamics before they can be modeled, so the trade-off remains unlearned.

To address this gap, we present BRACE, a benchmarking framework for *CLM*s that evaluates functional correctness, energy consumption, and model scale together. Rather than merging these competing dimensions into one scalar, BRACE decouples the evaluation into three pairwise profiles, namely energy–accuracy, energy–size, and accuracy–size, and maps each onto an interpretable 1–5 scale over 33 *CLM*s. First, to evaluate the empirical trade-off between energy and accuracy, we frame it as a Multi-Criteria Decision Making (MCDM) [111] problem and introduce two rating techniques. Concentric Incremental Rating Circle (***CIRC***) is a distance-based approach that measures each model's inefficiency by its distance to the best achievable objectives. Observation to Expectation Rating (***OTER***) is a parametric model that estimates the empirical energy–accuracy relation and rates each model by comparing its observed performance against the performance expected at its efficiency level. We assign each model a rating from 1 to 5, where 5 denotes jointly strong accuracy and efficiency and 1 denotes an energy-hungry, low-performing model. Second, to reduce the bias toward large models and to account for real-world deployment constraints, we build on neural scaling laws [56] for the accuracy–size and energy–size relations. By fitting a trend curve across the model set, we measure how much

accuracy a model extracts with respect to its resource consumption (*Performance Efficiency*) and whether it uses less energy than its footprint predicts (*Structural Efficiency*), and we map each onto a 1–5 rating. This rewards architectures and training choices that reach high utility through efficiency rather than through added parameters or excess energy. The five-point scale balances enough granularity to separate models against practical interpretability. Consistent with established energy rating frameworks [8, 33, 64], this rating is analogous to an appliance energy label but remains performance-aware, conveying at a glance where a model stands on the efficiency-performance frontier.

While BRACE applies broadly to language models, we focus on *CLM*s as an ideal testbed for two reasons. First, coding tasks demand *strict correctness*, unlike natural language generation, where semantic variation is acceptable, code must compile, execute, and pass tests, which enables objective, deterministic evaluation. Second, *CLM*s require *cross-domain projection*, since in code generation they map natural language specifications to executable code, while in code summarization they translate program semantics back to natural language explanations. This bidirectional translation between distinct formal and informal domains, rather than the *within-domain* generation typical of Natural Language Processing (NLP) tasks, creates unique computational demands that stress both the accuracy and efficiency of model architectures.

This study makes the following **contributions**.

- We propose two rating methods, ***CIRC*** and ***OTER***, that jointly rate LLMs on functional correctness and energy efficiency on a single interpretable scale, and we extend both with priority weighting so that users can encode accuracy-first or energy-first preferences.
- We introduce two scaling-law-based ratings, *Performance Efficiency* and *Structural Efficiency*, that measure how much accuracy a model extracts from its size and whether it uses less energy than its size predicts, which counters the bias toward large models and exposes where a model spends parameters without return.
- We present BRACE, a benchmarking framework for *CLM*s on SE tasks that unifies these ratings, supports systematic model profiling, and provides a basis for further work on sustainable AI for software engineering.
- We release BRACE [10], including code and data for replication and for benchmarking new models.

2 Background

As explained in Section 1, no suitable metric has been proposed that accounts for efficiency, energy, and performance simultaneously while providing a reliable overall score for all data points. We use accuracy and energy consumption here to show how existing metrics behave under specific conditions and where they fail to capture the desired trade-offs; the same reasoning extends to model size, which can enter as an additional objective or replace either accuracy or energy consumption.

Before diving into the explanation of the metrics, we reformulate our problem to a standard optimality problem, where we aim to maintain both aspects at high level without compromising any one of them. Also, in order to keep both aspects in the same bounded ratio and avoid unstable computations, we perform our operation on the normalized version of the reported values. To this end, for energy consumption, a lower usage corresponds to a higher value. Hence, we define the *energy efficiency* as eff_{norm} term to address this issue. We formulate this term as follows, where e represents a model's energy consumption of a task:

$$e_{norm} = \frac{e - e_{\min}}{e_{\max} - e_{\min}} \tag{1}$$

$$eff_{norm} = 1 - e_{norm} \tag{2}$$

However, for accuracy, as we expect to achieve high accuracy, and they are already aligned in this direction, we only apply min-max scaling on accuracy values. The following equation shows how we derive acc_{norm} where acc represents a model's accuracy on a task:

$$acc_{norm} = \frac{acc - acc_{\min}}{acc_{\max} - acc_{\min}} \tag{3}$$

Pareto Optimal

Pareto frontier is defined when multiple points can be considered as the most optimal points where none of the solution in Pareto frontier set can be outperformed by any other one. In other words, improvement in one objective worsen the other objective [79]. In mathematic terms, we can say x^* is pareto-optimal if:

$$\nexists\, y \in \mathcal{X} :\ (A(y) \geq A(x^\star)\ \wedge\ E(y) \geq E(x^\star)) \wedge\ \big(A(y) > A(x^\star)\ \vee\ E(y) > E(x^\star)\big). \tag{4}$$

where A is accuracy and E is energy efficiency in our problem. The Pareto frontier faces three limitations when it must produce a final score from two objectives. First, it identifies only the dominant points and leaves all others unnoticed. Second, because it does not combine both aspects into a single score, it cannot compare points. Third, points on the frontier can differ widely in final standing, because two points that differ substantially on one objective yet are similar on the other can lie on the same frontier. This follows from the frontier depending only on greater-or-lesser comparisons, without quantities or functional relations.

As a motivating case study, we use the ZEUS leaderboard energy and accuracy data [25], shown in Figure 1, to plot the Pareto-optimal models. The frontier marks five points as dominant, treating all of them as the most optimal models in the same class. As one example, points A and B have almost the same accuracy, with B slightly lower, yet B's energy efficiency is substantially higher than that of B. Hence, treating them as the same class would not be fair to B.

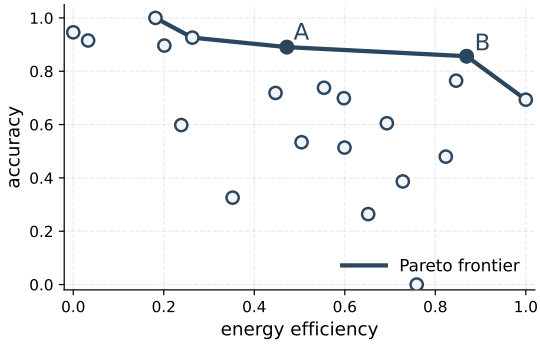

Fig. 1. Example of ZEUS data on two points a and b where it will assign a same label to them unfairly.

Mean Variants

The *arithmetic mean* is a common operation used when two equally weighted objectives should be maximized at once. By maximizing their sum, it defines the final score as:

$$Score = \frac{eff_{norm} + acc_{norm}}{2}$$

Despite its simplicity and wide use, this metric does not capture the underlying relationship between the two aspects. The arithmetic mean assumes a strict $1:1$ substitution rate between efficiency and accuracy, that is, a linear trade-off with slope -1. When the real relationship follows a different slope or is non-linear, the metric becomes skewed.

An alternative frames the problem as a performance-to-cost ratio [24], treating accuracy as performance and energy consumption as cost. On normalized values, the score becomes a simple ratio:

$$Score = \frac{acc_{norm}}{e_{norm}}$$

Much like the arithmetic mean, this ratio assumes a strict linear trade-off between accuracy and energy consumption, specifically a proportional relationship through the origin.

Finally, the *harmonic mean* is another option, widely used in machine learning, with the F1-score as its best-known example. We define it here as:

$$Score = \frac{2 \times eff_{norm} \times acc_{norm}}{eff_{norm} + acc_{norm}}$$

The harmonic mean scores models through a joint relation between the two objectives, but its scoring works regardless of objective correlation and penalizes a low value in either objective too aggressively, which can undermine strong performance in the other aspect.

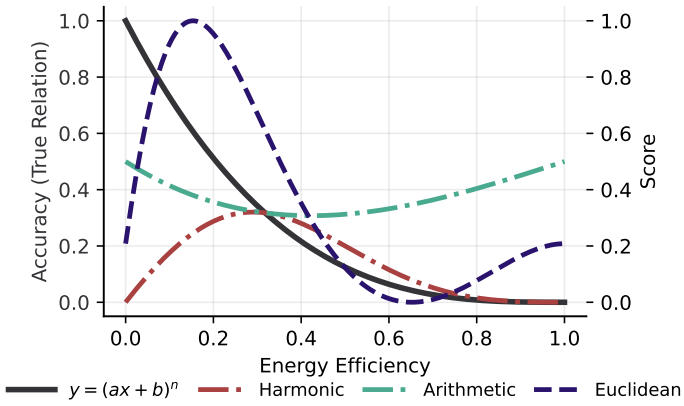

Fig. 2. Example of different mean-variant behaviors on a hypothetical relation between energy efficiency and accuracy.

Figure 2 shows how the mean-variant scoring methods behave. Considering a scenario where model accuracy and energy consumption can be represented by a polynomial relationship, $y = (ax + b)^n$, where x is energy and y is accuracy. Unlike a linear assumption, this synthetic non-linear relationship allows changes in energy efficiency to produce disproportionate gains or losses in accuracy, so the example shows how scoring methods respond under realistically possible non-linear trade-offs. The left y-axis gives the accuracy at each energy efficiency, where only $y = (ax + b)^n$ belongs to the left axis, and the right y-axis gives the scores of the mean variants and the Euclidean distance. The endpoints $(0, 1)$ and $(1, 0)$ are the most favorable to the arithmetic mean, yet a fair and robust system should not assign them the highest score. The harmonic mean instead penalizes low values in each objective and maximizes its score when both objectives are close and at a reasonable distance from zero, so it peaks where the arithmetic mean is weakest.

Its penalization is strict enough that it does not allow relative compensation between objectives. A fair and robust metric, therefore, needs to either model the joint relation between energy and accuracy or score based on a reasonable trade-off between the arithmetic and harmonic means, which motivates the methods in Section 3.

3 Study Design

Our goal is a scoring framework that evaluates *CLM*s across three competing dimensions, namely *functional correctness* (accuracy), *energy consumption*, and *model size*. Formally, this is a Multi-Criteria Decision Making (MCDM) [111] problem, where a finite set of alternatives is evaluated against criteria that conflict, so that no alternative is best on every criterion at once. The alternatives are the models under evaluation. The criteria are one benefit, accuracy (A), which a model should maximize, and two costs, energy consumption (E) and model size (S), which it should minimize. The three conflict because accuracy is ordinarily bought with both energy and scale. Our objective is not to select a single winner but to place every model on an ordered scale.

Prior approaches [33, 39] aggregate all three criteria at once into a single scalar, $s = f(A, E, S)$. Two properties of that design motivate our choices. First, a three-way aggregate cannot be attributed. One number cannot say whether a model is penalized for carrying parameters it fails to convert into accuracy or for drawing more energy than its accuracy justifies, so the score marks a model as weak without locating the weakness. Second, no pairwise trade-off survives the aggregation. Between two criteria, the rate at which one is exchanged for the other is a single quantity that can be plotted, inspected, and reported. A function of all three criteria offers no such quantity, since only two cases are possible. If the energy-accuracy rate varies with model size, no single rate exists to report. If instead it stays constant across model size, the form has assumed that energy and accuracy trade off the same way at every scale, and it never states that assumption. Either way, a reader cannot recover what the score treats as an acceptable exchange.

BRACE therefore aggregates only within a pair. Aggregation itself is not the flaw, since each of the three ratings is still a single number, as any rating must be. What a pair buys is a trade-off narrow enough to state, to inspect, and to attribute to the two criteria that produce it. A trade-off that small can be handled in either of the two ways Section 2 identifies, namely by modeling the relation between the criteria or by fixing a rule that balances the extremes, and under both the resulting exchange stays visible. Reporting the three ratings side by side keeps every dimension traceable, and a user who wants one number can combine them under weights they choose and can inspect.

3.1 Problem Statement

Let $M = \{m_1, m_2, \ldots, m_k\}$ be a set of models and $B = \{b_1, b_2, \ldots, b_{k'}\}$ a set of SE benchmarks. Each model $m \in M$ runs on every benchmark $b \in B$. From these runs, we collect the energy consumption profile of each model on three core devices, $D = \{cpu,\ gpu,\ ram\}$, together with its accuracy on a given benchmark. We obtain the energy consumption of a model on a benchmark as:

$$E_{m,b} \approx \sum_{d \in D} \sum_{n=1}^{N_{m,b}} P_{d,n}^{(m,b)}\, \Delta t, \quad \forall m \in M,\ b \in B \tag{5}$$

where Δt is the sampling interval in seconds, $P_{d,n}^{(m,b)}$ is the instantaneous power of a device in Watts, and $N_{m,b}$ is the total sampling count.

Let $A_{m,b} \in [0, 1]$ be the raw accuracy of model m on benchmark b, and let $S_m \in \mathbb{R}^+$ be the physical memory footprint that defines the model's scale. For a benchmark b, let $A_b = \{A_{m,b}\}_{m \in M}$ and $E_b = \{E_{m,b}\}_{m \in M}$ hold the accuracy and energy distributions across the model set.

Our goal is a multi-dimensional, discrete profile vector for each model-benchmark pair:

$$R_{m,b} = \begin{bmatrix} R^1_{m,b} \\ R^2_{m,b} \\ R^3_{m,b} \end{bmatrix} \in \{1, \ldots, 5\}^3 \tag{6}$$

where each coordinate is a 1–5 score from best (5) to worst (1). We define three evaluation functions that map the raw metrics into this scale:

(1) **Operational Energy-Accuracy Rating (R^1).** A joint rating function φ that rates a model's energy-accuracy trade-off relative to the distribution of the model set:

$$R^1_{m,b} = \varphi_{\text{acc-ene}}(A_{m,b}, E_{m,b} \mid A_b, E_b) \tag{7}$$

(2) **Performance Efficiency Rating (R^2).** A function ψ that measures how efficiently a model converts its size into accuracy, by comparing its observed accuracy against the accuracy expected at its size within the model set:

$$R^2_{m,b} = \psi_{\text{acc-size}}(A_{m,b}, S_m \mid A_b, S) \tag{8}$$

(3) **Structural Efficiency Rating (R^3).** A function τ that measures a model's energy use against the energy expected for its size:

$$R^3_{m,b} = \tau_{\text{ene-size}}(E_{m,b}, S_m \mid E_b, S) \tag{9}$$

By separating these ratings, the framework rewards architectures that reach strong performance without extreme energy use or added parameters, which supports transparent model selection for SE.

3.2 Research Questions

This study rates *CLM*s on their combined operational efficiency (energy and size) and functional correctness (accuracy). We structure the study around four Research Questions (RQs).

> **RQ1:** How can we benchmark *CLM*s effectively based on their energy-accuracy trade-offs using a unified rating method?

Comparing models on their raw energy-accuracy profiles is hard when trade-offs exist, because one model may reach higher accuracy at much higher energy while another favors efficiency at the cost of precision. Unlike parameter scale, whose effect on resources is mathematically predictable, the relation between task accuracy and hardware energy consumption has to be estimated empirically and scored fairly and robustly. Here, fairness means moderately performing models are not penalized, and robustness means models are not over-rewarded for easy gains.

> **RQ2:** How do our proposed rating methods evolve and maintain stability as new models are added to the benchmark?

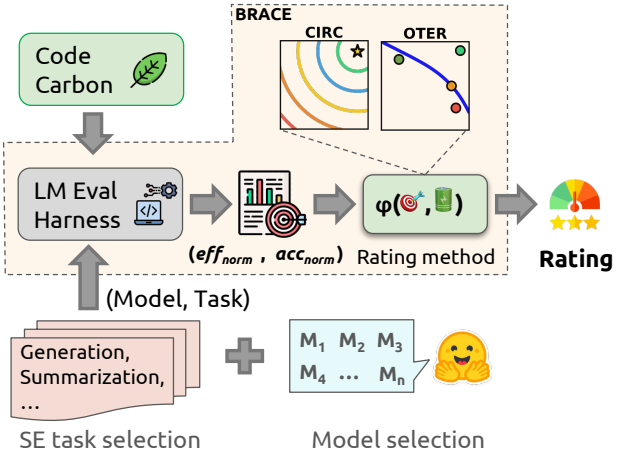

Fig. 3. Overview of the BRACE benchmarking framework. eff_{norm} and acc_{norm} are the normalized values of energy efficiency and accuracy of all models. TODO ▶ *@javad, im working on the new diagram*◀

Ranking stability matters for any new MCDM method, since benchmarking is an evolving process rather than a one-time measurement. Because the 1–5 scale maps ratings to the relative distributions of raw metrics, adding models shifts the existing boundaries. A benchmark that is meant to grow must reflect new entrants while re-scoring existing models against the updated distribution in a predictable way. This RQ examines how the distributions evolve, so that any rating change is justified and the ordering of the original models is preserved.

> **RQ3:** How effectively can our proposed rating methods integrate dynamic weighting to capture user priorities in accuracy- or energy-first deployment environments?

Equal weighting on energy and accuracy is a reasonable default, but deployments often impose asymmetric priorities. A mobile-edge coding assistant needs strong energy efficiency, while a cloud-based code reviewer needs accuracy. This RQ studies continuous weighting parameters (w_a, w_e) that let users control the relative importance of accuracy and energy, and it asks whether these weights steer the trade-off toward a stated priority while keeping the 1–5 ratings monotonic and interpretable.

> **RQ4:** To what extent do *CLM*s efficiently translate their physical parameter scale into predictive accuracy and energy efficiency?

Model size is the third dimension that decides whether a model can be adopted, alongside accuracy and energy. It carries a deployment cost, because an accurate and efficient model may still exceed the memory available on consumer-grade hardware, and a model may simply be larger than its performance warrants. It also distorts comparison, because ranking models of very different scales can hide architectural gains behind raw parameter count. This RQ measures how efficiently *CLM*s turn their parameter budget into accuracy and into energy savings, and it derives ratings that allow direct comparison across models.

4 Methodology

To address the research questions discussed in Section 3, BRACE evaluates *CLM*s along energy efficiency, task accuracy, and size-adjusted capability, following the workflow in Figure 3. It logs these metrics automatically across the selected SE tasks, and its scoring methods project the resulting multidimensional relations onto a single interpretable 1–5 scale, so that sustainability and architectural density become the primary criteria for model evaluation.

4.1 Benchmark dataset selection

Our primary criterion for selecting benchmarks is their relevance to SE tasks. We select two tasks to analyze model behavior across different problem types. We focus on generative tasks that are widely used in SE research to capture each model's true capability and resource usage.

Our first task is *code generation*, which assesses a model's ability to generate code that passes predefined test cases. We choose the *LiveCodeBench* benchmark [50], a recent and widely cited dataset that includes diverse code intelligence tasks [2]. We use the `pass@1` metric to assess models on this benchmark; the metric captures whether the model generates a correct solution on its first attempt [23]. Our second task, *code summarization*, involves generating a docstring for a given code block to assess the model's code comprehension ability [96]. We use the CodeXGLUE benchmark [62], a widely adopted dataset [3, 4, 52]. We adopt its Python code summarization variant and use smoothed BLEU [61] as the evaluation metric, following the original benchmark setup [62]. The two tasks complement each other by covering the two paradigms of code intelligence, text-to-code (generation) and code-to-text (comprehension). Because each task needs task-specific prompting, we evaluate several prompts following established prompt-engineering practice [11] and keep the prompt with the highest performance. The prompts can be found in our replication package [10].

4.2 Model selection

We use HuggingFace [29], the largest platform for open-source LLMs, for our model selection search. Given the SE focus, we evaluate only *CLM*s, and we consider open-source *CLM*s so they can be run locally for accurate energy measurement. Our initial search criterion selects models whose names include the term *"code"*, reducing the pool from 2,105,378 to 24,840. We then filter for *text-generation* models, narrowing the list to 8,840. Our computing environment (*i.e.,* a 32GB GPU) supports models up to 9 billion parameters at full 32-bit precision, so we filter out models with more than 9B parameters, which leaves 5,607 candidates. From this set, we exclude all quantized variants to ensure consistency and fairness and select the top 25 base models by download count. After a preliminary run, we exclude models that require custom dependencies beyond the standard HuggingFace implementations, trigger Out-of-Memory (OOM) CUDA errors, or show severe library incompatibilities, which yields a base set of 22 models. We later extend this base set to 33 models under the same protocol to study rating stability (Section 5.2). The list of the selected models and their download links can be found in our replication package [10].

4.3 Unified energy-accuracy scoring mechanisms

To address this MCDM problem, in which energy and accuracy must be considered jointly to assign a single score reflecting a model's overall efficiency and predictive performance, we adopt the terminology and metrics defined in Section 2. Because our goal is to maximize both objectives, *i.e.,* accuracy and energy efficiency, at once, we focus on acc_{norm} and eff_{norm}, and we introduce two rating methods that we elaborate next, unifying these normalized measures into a single scoring framework.

Concentric Incremental Rating Circle (CIRC). TODO ▶ *Javad: thanks @tushar, i tried to keep text as short as possible, but we need a introductory paragraph why Euclidean distance is our choice.*◀ Mean-based variants provide a potential way to jointly account for energy and accuracy, but as Section 2 and Figure 2 show, they are either overly lenient or excessively stringent. In contrast, euclidean distance balance a sound trade-off between arithmetic and harmonic mean, favoring moderate compensation between the objectives.

We propose ***CIRC***, which builds on Euclidean distance and draws on TOPSIS [49], an MCDM algorithm that ranks options by their Euclidean distance to the ideal and anti-ideal solutions. In the first step of ***CIRC***, we fix the optimal points. Unlike TOPSIS, which sets these points relative to the data, we use the normalized nature of our data to fix them absolutely. Because both objectives are normalized within $0 \le o_1, o_2 \le 1$, we set the best point at $(1, 1)$, for maximum accuracy and energy efficiency, and the worst point at $(0, 0)$, which are $\sqrt{2}$ apart in Euclidean distance.

To rate models on a 1–5 scale, we partition the interval $[0, \sqrt{2}]$ into five equal segments of length $\frac{\sqrt{2}}{5}$, which serve as score boundaries. Each boundary is a circle, so all points on it are equidistant from the ideal, using $(x-c_x)^2+(y-c_y)^2 = r^2$. ***CIRC*** imposes no monotonic trade-off between energy and accuracy, but because of the circular formulation, scores stay invariant when energy and accuracy change along a circle, which reflects ***CIRC***'s notion of their relative variation. This construction gives five concentric circles centered at $(1, 1)$, with radii increasing in equal increments from $\frac{\sqrt{2}}{5}$ to $\sqrt{2}$.

We assign the highest score to a model inside the first circle, the one nearest the ideal. Models with distances between the first and second radii receive four, and so on until the region between the fourth and fifth circles, which receives one. We define the ***CIRC*** rating as:

$$
\begin{aligned}
d_i^2 &= (x_i - 1)^2 + (y_i - 1)^2 \\
r_k &= k\,\frac{\sqrt{2}}{5}, \quad k \in \{1, \ldots, 5\} \\
R_i^1 &= 6 - \min\{\, k \in \{1, \ldots, 5\} : d_i \le r_k \,\}
\end{aligned}
\tag{10}
$$

where d_i is the model's distance to the ideal and the r_k are the five boundary radii. The rating selects the nearest enclosing circle, so a model at the ideal falls inside r_1 and scores 5, a model at the maximum distance $\sqrt{2}$ falls only inside r_5 and scores 1, and $R_i^1 \in \{1, \ldots, 5\}$ throughout.

Weighted CIRC for Priority Elasticity. The baseline ***CIRC*** weights energy efficiency and accuracy equally ($w_a = w_e = 0.5$). Deployments often need asymmetric priorities. In an energy-constrained setting, a practitioner may weight efficiency at 0.8 and accuracy at 0.2, and at the extreme may evaluate on a single axis and neutralize the other.

We define w_a and w_e as continuous priority weights for accuracy and energy efficiency, with $w_a + w_e = 1$, and formulate the weighted distance as:

$$
d_i = \sqrt{2w_a(1 - y_i)^2 + 2w_e(1 - x_i)^2} \quad , \quad \text{s.t.} \quad w_a + w_e = 1 \tag{11}
$$

where the Likert rating follows from d_i as in the baseline. The factor of 2 makes the weighted distance recover the baseline Euclidean distance at $w_a = w_e = 0.5$, and it keeps the maximum distance from $(0, 0)$ equal to $\sqrt{2}$ for every weighting, so the $[0, \sqrt{2}]$ partition remains valid throughout.

The formulation inherently considers geometric elasticity. At $w_a = w_e = 0.5$ the boundaries are concentric circles. As the user shifts priority toward energy ($w_e > w_a$), the circles deform into vertical ellipses, which raises the marginal penalty for energy inefficiency and loosens the penalty for accuracy loss. In the extreme $w_e = 1, w_a = 0$, the ellipses

degenerate into vertical lines along the x-axis at $\{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$, so accuracy is neutralized and models are binned purely by energy efficiency, which matches single-objective demands.

Observation to Expectation Rating (OTER). The second approach we propose is ***OTER*** to address the linearity assumed by mean-based and ratio-based methods. These methods implicitly assume that an increase in energy should bring a proportional, linear increase in accuracy to maintain balance. However, this assumption does not hold, because the relation between energy and accuracy is often non-linear and varies across models [7, 54]. Improvements in accuracy do not require energy to grow at the same rate [7, 25], so a method needs to capture this compensation fairly and dynamically from observed patterns.

OTER is inspired by Stochastic Frontier Analysis (SFA) [5], a parametric method that models input-output relations to define an optimal frontier, but we tailor it to our setting. We first postulate that as models become more energy efficient, a degree of accuracy compromise is acceptable [16, 112], a pattern also seen in Pareto-front analyses [7, 48]. This assumption lets us reward models that improve both objectives relative to weaker models and learn the energy-accuracy trade-off rather than ignore it. We place energy efficiency on the x-axis and accuracy on the y-axis, and we fit a monotonically decreasing reference curve to the dominant empirical trend, unlike SFA, which estimates a frontier of top performers. This curve gives an interpretable notion of the acceptable trade-off, that is, the expected accuracy at each efficiency level.

A model is then rewarded or penalized by how its accuracy compares to the expected level, measured as the ratio of actual to expected accuracy. A model above the curve exceeds the accuracy expected at its efficiency level and is rewarded, while a model below the curve falls short and is penalized for converting energy into accuracy inefficiently relative to the dominant behavior. We rate models on a 1–5 scale by dividing the range of the actual-to-expected ratio into five equal intervals and assigning each model to its interval. We formulate this as:

$$
\begin{aligned}
&\mathcal{X} = \{(x_i, y_i)\}_{i=1}^{n}, \\
&\arg\min_{f \in \mathcal{F}} \sum_{i=1}^{n} \ell\left(y_i, f(x_i)\right) \quad \text{s.t.} \quad f(x) \ \textit{decreasing} \\
&v_i = \frac{y_i}{f(x_i)}, \qquad v_{\min} = \min_{1 \le i \le n} v_i, \quad v_{\max} = \max_{1 \le i \le n} v_i, \\
&\Delta = \frac{v_{\max} - v_{\min}}{5}, \qquad R_i^1 = \max(1, \lceil \frac{v_i - v_{\min}}{\Delta} \rceil)
\end{aligned}
\tag{12}
$$

where $\mathcal{X}$ is the set of energy (x) and accuracy (y) pairs for all models, f is a monotonically decreasing function that best fits the data under objective ℓ, and a model's raw score v_i divides the observation y by the expectation $f(x)$. Δ is the interval width per class, and the rating R follows from the interval a model's score falls into.

Figure 4 outlines the rating approach. ***OTER*** favors models that maintain or improve accuracy as energy efficiency increases, which ensures fairness, because a model that beats another on both objectives should score higher. The

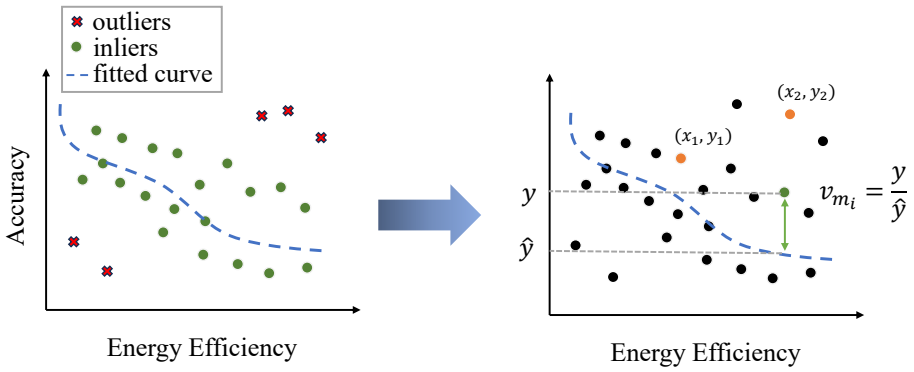

Fig. 4. Overview of the ***OTER*** approach. We first fit a curve to the majority of the points, then compute a raw score v as the ratio of a model's observed to its expected accuracy.

monotonically decreasing curve gives this property. For two points (x_1, y_1) and (x_2, y_2), this is:

$$
\begin{aligned}
& f(x) \text{ decreasing}, \quad V(x,y) = \frac{y}{f(x)}, \quad y_2 \geq y_1 \geq 0. \\
& x_2 > x_1 \;\Rightarrow\; f(x_2) < f(x_1) \\
& \Rightarrow\; \frac{1}{f(x_2)} > \frac{1}{f(x_1)} \qquad (f(x) > 0) \\
& \Rightarrow\; \frac{y_2}{f(x_2)} > \frac{y_1}{f(x_1)} \;\Rightarrow\; V(x_2, y_2) > V(x_1, y_1).
\end{aligned}
\tag{13}
$$

For models whose accuracy decreases as energy efficiency increases, the raw score depends on the curve and the deviation from it. Points above the curve score above 1, points below score below 1, and points near the curve score near 1, at the center of the distribution. Models that deviate most receive the highest or lowest scores, which reflects an exceptionally strong or weak energy-accuracy trade-off. Like ***CIRC***, ***OTER*** imposes no fixed energy-accuracy correlation and avoids static assumptions by estimating the compensation from observed data, using a monotonically decreasing curve only to allocate reward fairly. Equation 12 requires two components, namely an optimization method that minimizes the error ℓ under the decreasing-monotonicity constraint, and the function $f(x)$.

For the optimization, we use quadratic programming with linear constraints, because it enforces monotonicity strictly, unlike genetic or learning-based methods, and fits smooth functions, unlike isotonic regression, which yields step-wise solutions [42, 92]. At least one decreasing solution exists regardless of its proximity to the data, so quadratic programming can minimize prediction error while controlling curvature and monotonicity through linear constraints.

We model $f(x)$ as a polynomial for three reasons. Polynomials approximate bounded regions ($0 \leq x, y \leq 1$) well [14], a property the Stone-Weierstrass theorem guarantees [108]. They encode monotonicity and convexity naturally, which supports the solutions required in quadratic programming [58]. They are also established in the optimization literature [13, 58, 95].

Because the sample is limited and a few outliers can distort the fit, we remove outliers as an auxiliary step (shown in Figure 4). We compute Mahalanobis distances from a Minimum Covariance Determinant (MCD) [87, 88], which is robust to outliers [47] and correlation-aware [27]. We mark observations whose squared robust distance exceeds the chi-square threshold at the 95^{th} percentile with 2 degrees of freedom as outliers. We fix the variables that control the

monotonic shape of the curve. To fit a decreasing curve, every point on the curve must have a derivative below 0. When the data shows an atypical pattern, where higher energy efficiency coincides with higher accuracy for most points, the closest fitted function is a near-constant curve, which scores unfairly. If two models have similar accuracy but very different energy efficiency, a near-constant $f(x)$ gives them similar raw scores under $y/f(x)$, even though the more efficient model should score higher.

To ensure a fair and robust decreasing slope, we introduce the *Least Expected Slope (LES)*. We compute pairwise slopes for all point pairs that show an energy-accuracy trade-off, that is, negative slopes. Following systems benchmarking practice [9, 104], we use Q3, the third quartile of the computed slopes. Because the slopes are negative, this gives a conservative upper bound on the expected derivative, and we constrain the quadratic program to stay below *LES* across the range. We define *LES* as:

$$\begin{aligned} D &:= \left\{ d_{ij} := \tfrac{y_j - y_i}{x_j - x_i} \;:\; i \neq j,\; x_j \neq x_i,\; d_{ij} < 0 \right\} \\ LES &:= Quantile_{0.75}\big(MCD(D)\big), \quad LES < 0 \end{aligned} \tag{14}$$

We ensure the curve stays above zero, setting a constraint at $f(x = 1)$ because the function is monotonic, so its minimum is at $x = 1$. We also cap the maximum expected accuracy at one. To fit the best function, satisfy the *LES* constraint, and keep the curve in $[0, 1]$, we formulate the quadratic program as:

$$\begin{aligned} &\min_{\boldsymbol{\beta} \in \mathbb{R}^{p+1}} \sum_{i=1}^{n} \Big(y_i - \sum_{k=0}^{p} \beta_k x_i^k \Big)^2 \\ &\text{s.t.} \quad \sum_{k=1}^{p} k\, \beta_k\, x^{k-1} \;\leq\; LES \qquad \forall\, x \in [0, 1]. \\ &\qquad \sum_{k=0}^{p} \beta_k \;\geq\; \epsilon \quad s.t. \quad \epsilon = 1e^{-2} \\ &\qquad \beta_0 \leq 1 \end{aligned} \tag{15}$$

The first line minimizes the error between predictions and ground truth. The second line gives the linear constraints, which hold every derivative below *LES*. The third constraint keeps the curve above zero. The fourth constraint caps the intercept, and because f is decreasing on $[0, 1]$, its maximum is $f(0) = \beta_0$, so $\beta_0 \leq 1$ caps the expected accuracy at one. We use a polynomial degree of five to balance flexibility against stability, because higher degrees can cause instability [17] and lower degrees may not capture the variance.

Weighted OTER for Priority Elasticity. As with weighted ***CIRC***, we extend ***OTER*** to user-defined weights w_a for accuracy and w_e for energy efficiency, with $w_a + w_e = 1$. A weight sweep has to do more than tilt the score toward one objective. At each extreme it must reduce to a single objective on a linear scale, because only a linear scale splits into five equally wide bins, and equal bins are what keep the rating readable once the other objective is switched off. The difficulty lies in this fallback rather than in the tilt, and the rest of the construction is what secures it.

The starting point is the baseline score, which already factors into one term per objective:

$$v_i = \frac{y_i}{f(x_i)} = y_i \times \frac{1}{f(x_i)} \tag{16}$$

Accuracy enters directly through y_i, while energy efficiency enters only through $\frac{1}{f(x_i)}$, which increases with x because f decreases. Because the two terms multiply, ordinary scalar weights would merge into a single constant and change

nothing. The weights must instead act as exponents, following the Cobb-Douglas production function ($P = bL^kC^{1-k}$) [26], which is widely used to combine multiplicative factors in economic [66, 78] and time-cost [94] utility models.

Exponentiating the two terms directly gives a first attempt, $S_{\text{naive}} = y_i^a \times \left(\frac{1}{f(x_i)}\right)^b$, but it meets the requirement on only one side. Isolating accuracy (a=1, b=0) leaves $S = y_i$, which is linear, so equal score intervals give equally wide bins. Isolating energy (a=0, b=1) leaves $S = \frac{1}{f(x_i)}$, and because f is a degree-five polynomial, equal intervals of its reciprocal fall on unequal intervals of x, so the bins come out skewed and compressed. The asymmetry traces back to the baseline score, where accuracy is already a bare linear term and needs no repair, while energy efficiency sits inside the non-linear curve and needs a linear term of its own.

We therefore keep the ratio $\frac{y}{f(x)}$ intact as a single efficiency factor and inject an explicit linear term for whichever objective the user favors, which gives a piecewise score:

$$S(x,y) = \begin{cases} y^{2w_a-1}\left(\dfrac{y}{f(x)}\right)^{2w_e}, & \text{if } w_a > w_e \\ x^{2w_e-1}\left(\dfrac{y}{f(x)}\right)^{2w_a}, & \text{otherwise} \end{cases} \tag{17}$$

$$\text{s.t.} \quad w_a + w_e = 1, \quad w_a, w_e \in [0,1]$$

Substituting $w_a = 1 - w_e$ shows what each branch computes:

$$S(x,y) = \begin{cases} y \times \left(\dfrac{1}{f(x)}\right)^{2w_e}, & \text{if } w_a > w_e \\ x^{2w_e-1} \times \left(\dfrac{y}{f(x)}\right)^{2w_a}, & \text{otherwise} \end{cases} \tag{18}$$

When accuracy dominates, the exponents on y cancel to one, so accuracy stays linear and the weight $2w_e$ sets only how strongly the trade-off curve bends it. When energy dominates, the factor x^{2w_e-1} supplies the linear term the naive form lacked, while the exponent $2w_a$ fades out the curve and suppresses accuracy at the same time. The tilt is monotone across the range, since the relative pull of efficiency over accuracy, $\frac{\partial S/\partial x}{\partial S/\partial y}$, grows as w_e rises from 0.5 to 1.

Evaluating the reduced form at the two extremes and the midpoint confirms the fallback requirement:

$$S(x,y) = \begin{cases} y, & w_a = 1 \quad \text{(accuracy only)} \\ \dfrac{y}{f(x)}, & w_a = w_e = 0.5 \quad \text{(baseline)} \\ x, & w_e = 1 \quad \text{(energy only)} \end{cases} \tag{19}$$

At the two extremes the score collapses onto a single linear axis, so the bins sit at $\{0.0, 0.2, \ldots, 1.0\}$ and match the degenerate behavior of weighted ***CIRC***. At the midpoint both branches return the unweighted score v_i, so the seam between them is continuous and the weighting reduces to the baseline whenever the user states no preference. Figure 5 shows this deformation, from accuracy-set horizontal bands, through the balanced trade-off, to energy-set vertical bands.

4.4 Size Efficiency of Models

Model size is the third axis of a model's real-world cost, alongside accuracy and energy, since a model that is both accurate and efficient can still be too large to deploy or simply larger than its results justify. Rating accuracy and energy

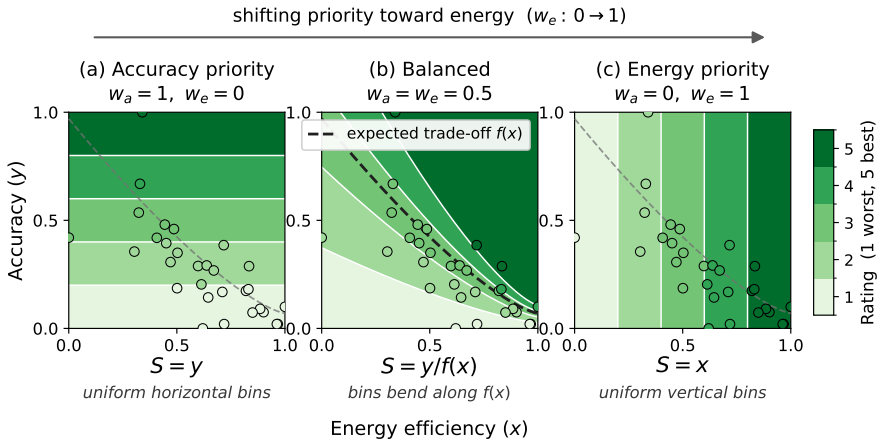

Fig. 5. Priority elasticity of weighted ***OTER***. As the energy weight w_e rises from 0 to 1, the rating field rotates from horizontal bands set by accuracy alone, through curved bands that follow the expected trade-off $f(x)$ at balanced weights, to vertical bands set by energy efficiency alone. The two extremes give uniform, equally wide bins, while the balanced field bends along $f(x)$. Points are an illustrative cohort, and shading gives the 1–5 rating.

alone leaves this axis unmeasured, so a large model and a smaller one with the same profile score alike even though the smaller design earns its parameters better. The energy-accuracy trade-off has no established law, which is why it needs the two rating methods above, whereas size relates to capability through well-studied empirical Neural Scaling Laws [56], where downstream performance and resource consumption [30] grow predictably with model size. We build these laws into two ratings of how efficiently a model turns its size into accuracy and into energy savings. Because models of similar size still differ widely in accuracy and energy for architectural and implementation reasons [7, 19, 48], a model's distance from the size trend is itself a signal of how well it uses its footprint.

We therefore fit one cross-model curve to the size-accuracy relation and another to the size-energy relation, each capturing the global trend and setting the accuracy or energy expected at a given size. A model's rating then reflects how far it beats or misses that trend for its footprint, following the same observed-against-expected intuition as ***OTER***. Fitting one global trend rather than a separate curve per model family is standard, since diverse architectures share a common scaling trajectory and differ mainly in how efficiently they convert resources into capability [67, 89], and a positive deviation from a single universal trend reads as an architectural or algorithmic advantage [80]. The same reading applies to energy, where a model off the trend either wastes resources or manages them well. We use model size rather than compute (FLOPs) as the scaling variable, since scaling laws stay structurally invariant across that choice, whether the parameter is training compute, model size, or tokens [18, 56].

We measure size S_m as a model's operational *memory footprint in Bytes* rather than its raw parameter count, because memory reflects the precision format (for example FP32, FP16, or FP8). A 7B model in FP32 occupies twice the space of its FP16 counterpart and carries more information per parameter. Memory footprint also sets deployment feasibility, since hardware limits such as GPU VRAM act as hard boundaries on model selection.

We now operationalize the two functions ψ and τ from Section 3. Both compare a model's observed metric against the value its size predicts, an idea related to the "Capability Density" of Xiao *et al.* [105], which measures the utility packed into each unit of a model's footprint. Let $\hat{A}(S_m)$ and $\hat{E}(S_m)$ be the accuracy and energy expected for a model of

size S_m. We define Performance Efficiency (R^2) and Structural Efficiency (R^3) as:

$$R^2_{m,b} = \mathcal{D}\left(\frac{A_{m,b}}{\hat{A}(S_m)}\right) \qquad R^3_{m,b} = \mathcal{D}\left(\frac{E_{m,b}}{\hat{E}(S_m)}\right) \qquad \text{s.t.} \quad R^2_{m,b}, R^3_{m,b} \in \{1, \ldots, 5\} \tag{20}$$

where $\mathcal{D}$ uniformly discretizes each ratio into a 1–5 scale over the observed range. The two references share this fitted-trend idea but are built differently. The energy reference $\hat{E}$ is the size-energy trend itself, because that relation does not saturate. The accuracy reference $\hat{A}$ needs one adjustment first, because the size-accuracy trend flattens as accuracy nears its ceiling. We develop the accuracy reference next, then the simpler energy reference.

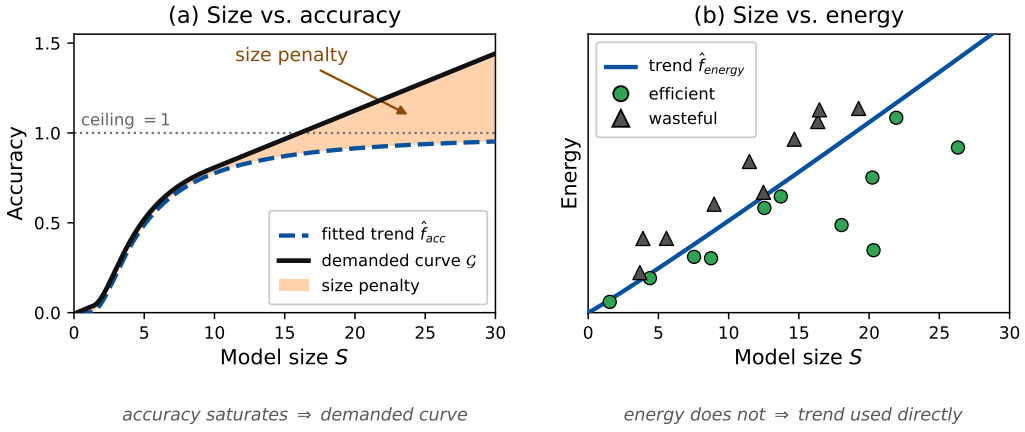

Fig. 6. Size-efficiency references (illustrative). (a) The size-accuracy trend $\hat{f}_{acc}$ saturates, so the demanded curve $\mathcal{G}$ tracks it while accuracy grows with size and keeps rising once it flattens, charging a model for size that buys no accuracy (shaded penalty). As a demand rather than a prediction, $\mathcal{G}$ can exceed the accuracy ceiling of 1, so a very large model cannot meet it. (b) The size-energy trend $\hat{f}_{energy}$ does not saturate and is used directly, so a model below it earns a high Structural Efficiency (τ) and a model above it a low one. TODO ▶ *@saurabh, thanks, but can you just make the trend-curve of size-energy a little more curve like not pure linear?* ◀

TODO ▶ *i follow up to this point more or less, not after this. can you zoom out and show what to expect next and why it is imp*◀

TODO ▶ *javad: so first, we only explain in very high-level why size is important and how size-based scaling laws for energy and accuracy for different architecture of LLMs can be applied. so it's high level design of our method. After this we will go into size-acc and size-energy in detail, because each has their own formula, we need to say how they work.*◀

Performance Efficiency Expectation Rating (Size vs. Accuracy). TODO ▶ *its unclear why it is needed, and whether we are defining it*◀ TODO ▶ *javad: the previous paragraph, we just justified how scaling laws can be used here, but we didn't talk about the formula and interpretation of size-acc and size-energy, here till the end, we discuss what approach we are taking to tackle each pair of size-acc and size-energy problem. AND yes we are defining the terms, but the primary equations of scaling laws were proposed earlier, we are modifying those scaling laws equations into our novel rating mechanism.*◀ Scaling laws were formulated for training-time validation loss [56], but they carry over to inference-time accuracy, which is strongly correlated with validation loss [28, 45]. Since downstream accuracy is bounded in $[0, 1]$, the size-accuracy curve must be S-shaped and monotonic [6, 80, 105]. We adopt the sigmoidal form of Krajewski *et al.* [57], which they define on the compute axis at

a fixed token-to-parameter ratio, and we take model size S as the scaling variable. It fits downstream accuracy more closely than earlier scaling-law forms and generalizes across settings once calibrated [18, 56, 57], mapping size $S \in \mathbb{R}^+$ to accuracy in $[0, 1]$ with monotonic S-shaped growth:

$$-\log(Q) = \frac{\beta}{S^\alpha} \implies \hat{f}_{acc}(S) = \exp\left(-\frac{\beta}{S^\alpha}\right) \tag{21}$$

where Q is the observed accuracy and $\hat{f}_{acc}(S)$ the accuracy predicted at size S (in GB). We fit β and α by ordinary least squares on the double-logarithmic form of Equation 21, shifting any model with zero accuracy by 10^{-5} to keep the logarithm defined.

Why the fitted trend alone is not enough. The obvious rating divides observed accuracy by the accuracy expected at that size, $v_i = A_i / \hat{f}_{acc}(S_i)$. Where the trend still rises, this already favors smaller models, since two models of equal accuracy are divided by different expectations and the larger one scores lower. The signal disappears once the trend flattens. Because accuracy is bounded, $\hat{f}_{acc}$ saturates, so beyond a certain footprint its slope approaches zero and two models of equal accuracy but very different size are measured against nearly the same expectation. A bloated model that merely matches a smaller one on the flat stretch then escapes penalty, which breaks the premise that added size must buy added accuracy. Recovering the size signal from the horizontal axis, as in a capability-density term $\hat{f}^{-1}_{acc}(A)/S$ in the spirit of Xiao *et al.* [105], does not help either, because where $\hat{f}_{acc}$ is flat its inverse is ill-conditioned and a negligible accuracy change maps to an unbounded change in implied size. We therefore keep the vertical ratio and repair the quantity it divides by.

The demanded curve. We need a reference that matches the trend while accuracy still grows with size, then keeps rising once the trend saturates, so that added parameters keep counting against a model unless they raise accuracy. This is a floor on the trend's slope. It defines the *demanded curve* $\mathcal{G}$, obtained by clamping the slope to a minimum rate μ and integrating from a small $S_0 > 0$:

$$\mathcal{G}'(S) = \max\big(\hat{f}'_{acc}(S),\ \mu\big), \qquad \mathcal{G}(S) = \hat{f}_{acc}(S) + \int_{S_0}^{S} \max\big(0,\ \mu - \hat{f}'_{acc}(t)\big)\, dt. \tag{22}$$

Where the trend climbs at least as fast as μ, the penalty term is zero and $\mathcal{G}$ tracks $\hat{f}_{acc}$. Where the trend flattens, $\mathcal{G}$ rises at rate μ and charges the model for size that no longer buys accuracy, as Figure 6(a) shows. Because $\mathcal{G}$ states what a model of a given size is asked to deliver rather than what it typically delivers, it serves as the accuracy reference $\hat{A}$ in Equation 20 and is not capped at one. By construction $\mathcal{G}$ is strictly increasing ($\mathcal{G}' \geq \mu > 0$) and never falls below the trend, so at equal accuracy a larger model always scores lower, and a model that dominates another on both accuracy and size never scores below it.

We set μ to the trend's average slope over $[0, S_{\max}]$, which equals $\hat{f}_{acc}(S_{\max})/S_{\max}$ because $\hat{f}_{acc}(0) = 0$. This makes the penalty self-adjusting. On a steep benchmark the local slope stays above μ, so $\mathcal{G}$ tracks the trend and the penalty stays mild, while on a saturating benchmark the slope drops below μ, the floor activates, and $\mathcal{G}$ keeps rising to penalize models that grow without a matching accuracy gain. The average gives a parameter-free default, and μ can be scaled up or down for a stricter or gentler penalty.

Performance Efficiency then rates each model by $v_i = A_i / \mathcal{G}(S_i)$, discretized into five classes over the observed range exactly as ***OTER*** does in Equation 12.

Structural Efficiency Expectation Rating (Size vs. Energy). Structural Efficiency mirrors this construction on the energy axis. A model should not draw more energy than the baseline for its size, and one that draws less is rewarded.

The rating is therefore size-relative, measuring energy discipline against the size expectation rather than absolute energy cost, which the energy-accuracy profile in RQ1 already captures. Unlike accuracy, energy does not saturate with size, so the fitted trend serves as the reference directly and needs no slope floor, as shown in Figure 6(b). Energy relative to size has been described as linear [30, 40] or log-linear [1, 32], so we model the baseline with a flexible power law that spans both:

$$E = \beta S_m^{\gamma} \tag{23}$$

where $\gamma = 1$ recovers the strictly linear case [30, 40]. We fit β and γ by least squares on the log-linear form $\log E = \log \beta + \gamma \log S$, and the resulting curve $\hat{f}_{energy}(S_m)$ is the energy reference $\hat{E}$ in Equation 20. We measure energy as the total consumed across the whole generation rather than per token, because a capable model may finish a task in fewer tokens, and less total energy, than a smaller model that struggles and over-generates. Total generation is the fair unit, since it charges for over-generation and captures true resource waste. Because energy is a cost, the rating inverts the deviation so that a model below the trend, using less energy than its size predicts, scores highest:

$$\begin{aligned} u_i &= \frac{E_i}{\hat{f}_{energy}(S_i)}, \quad u_{\min} = \min_{1 \le i \le n} u_i, \quad u_{\max} = \max_{1 \le i \le n} u_i \\ \Delta &= \frac{u_{\max} - u_{\min}}{5}, \qquad R_i^3 = 6 - \max\left(1, \left\lceil \frac{u_i - u_{\min}}{\Delta} \right\rceil\right) \end{aligned} \tag{24}$$

The inversion ($6 -$ term) makes $R_i^3 = 5$ the most structurally efficient architecture.

4.5 Experimental setup

Libraries: BRACE uses two open-source components, namely LM Evaluation Harness [35] and CodeGreen [85]. LM Evaluation Harness is a widely used benchmarking platform that supports applications such as the OpenLLM HuggingFace Leaderboard [34] and provides over 180 LLM benchmarks. CodeGreen is a low-overhead, hardware-agnostic energy profiling framework for precise, sub-millisecond software power measurement across diverse systems.

Energy Measurement Setup and Execution Protocol: We extract energy measurements across four stages, namely model configuration, tokenizer initialization, model initialization, and instance inference. We select four stages to match common LLM power benchmarking practice that separates setup overhead from pure *CLM* involvement [76], and because this four-tier granularity yields a reproducible protocol that generalizes across models without requiring intrusive, fine-grained phase profiling such as prefill versus decode.

We set CodeGreen's sampling interval to one second to balance sampling frequency against noise [86]. We validate CodeGreen's CPU readings against *perf* and find them consistent, and we ensure no other processes run during experiments. To ensure consistency, we execute each model-benchmark combination three times. All models retain default configurations with fixed random seeds for reproducibility, are offloaded to GPU during initialization, and run sequentially through identical scripts. No remote HuggingFace downloads occur during runs.

Hardware: Our primary machine features an RTX 5000 Ada GPU (32GB VRAM) and an AMD EPYC 9554P 64-core processor (755 GB). For reproducibility, we repeat experiments on a secondary machine with an Intel Xeon Gold 5317 CPU and an RTX 3070 Ti GPU (8 GB VRAM).

ID	Model	LCB		CXG		LCB Rating		CXG Rating	
		A	*E*	*A*	*E*	CIRC	OTER	CIRC	OTER
M_1	deepseek-coder-6.7b-base	0.23	0.31	0.47	0.23	2	1	2	3
M_2	starcoderbase-1b	0	0.93	0	0.88	2	1	2	1
M_3	starcoder2-3b	0.02	0.23	0.37	0.32	1	1	2	3
M_4	CodeLlama-7b-Instruct-hf	0.21	0.82	0.76	0.23	3	1	3	4
M_5	Qwen2.5-Coder-3B-Instruct	0.52	0.95	0.36	0.57	4	3	3	3
M_6	Qwen2.5-Coder-7B-Instruct	0.38	0.78	0.36	0.76	3	2	3	3
M_7	Qwen2.5-Coder-1.5B	0.48	0.99	0.31	0.88	4	3	3	3
M_8	deepseek-coder-1.3b-base	0.1	0.76	0.7	0.79	2	1	4	5
M_9	deepseek-coder-7b-instruct-v1.5	0.58	0.77	0.35	0.35	4	3	2	2
M_{10}	starcoder2-7b	0.04	0.15	0.56	0.21	1	1	2	3
M_{11}	codegen-350M-mono	0.02	0.83	0.09	0.83	2	1	2	1
M_{12}	Qwen2.5-Coder-0.5B	0.27	0.99	0.37	0.88	3	2	3	3
M_{13}	Yi-Coder-9B	0.33	0	0.16	0	1	1	1	1
M_{14}	rombos_Replete-Coder-Llama3-8B	0.21	0.74	0.2	0.28	3	1	2	2
M_{15}	speechless-code-mistral-7b-v1.0	0.31	0.85	0.37	0.6	3	2	3	3
M_{16}	stable-code-3b	0.06	0.68	0.53	0.67	2	1	3	4
M_{17}	CodeLlama-7b-Python-hf	0.15	0.73	0.38	0.23	2	1	2	2
M_{18}	Seed-Coder-8B-Instruct	1	0.88	0.31	0.59	5	5	3	2
M_{19}	CodeQwen1.5-7B-Chat	0.06	0.99	0.03	1	2	1	2	1
M_{20}	Magicoder-S-DS-6.7B	0.42	0.62	0.39	0.42	3	2	3	3
M_{21}	granite-8b-code-base-4k	0.1	1	1	0.14	2	1	2	5
M_{22}	codegen-2B-mono	0.02	0.64	0.45	0.51	2	1	3	3

Table 1. Summary of model performance on LiveCodeBench (LCB) and CodeXGLUE (CXG): normalized accuracy (*A*), normalized efficiency (*E*) and unified ratings (CIRC and OTER) for each benchmark. TODO ▶*changed rate to rating, the score numbers maybe need to change to center aligned for the ratings to get the green last column proper width, same for other table*◀

5 Results

5.1 RQ1: Energy-Accuracy Rating

In this section, we present the evaluation results of all models and compare their performance on the unified scale of energy-accuracy using our rating methods.

Comparative Analysis on Two Benchmarks: Table 1 reports accuracy and energy efficiency for all models on both tasks, together with the ***CIRC*** and ***OTER*** ratings and the model IDs. Seed-Coder-8B-Instruct (M_{18}) achieves a perfect score on LiveCodeBench and a moderate one on CodeXGLUE, making it the most optimal overall. This model, built on the Llama backbone, efficiently converts computational capacity into accuracy. Following it are the Qwen models (M_5 and M_7), attaining moderately high ratings on both tasks. They are the only models that do not sacrifice one objective for the other, maintaining a sound trade-off. In contrast, the largest Qwen model (M_6) performs worse than smaller variants on LiveCodeBench, highlighting that larger models do not guarantee higher accuracy. The weakest model, Yi-Coder-9B (M_{13}), shows the lowest energy efficiency and moderate accuracy, indicating inefficient parameter

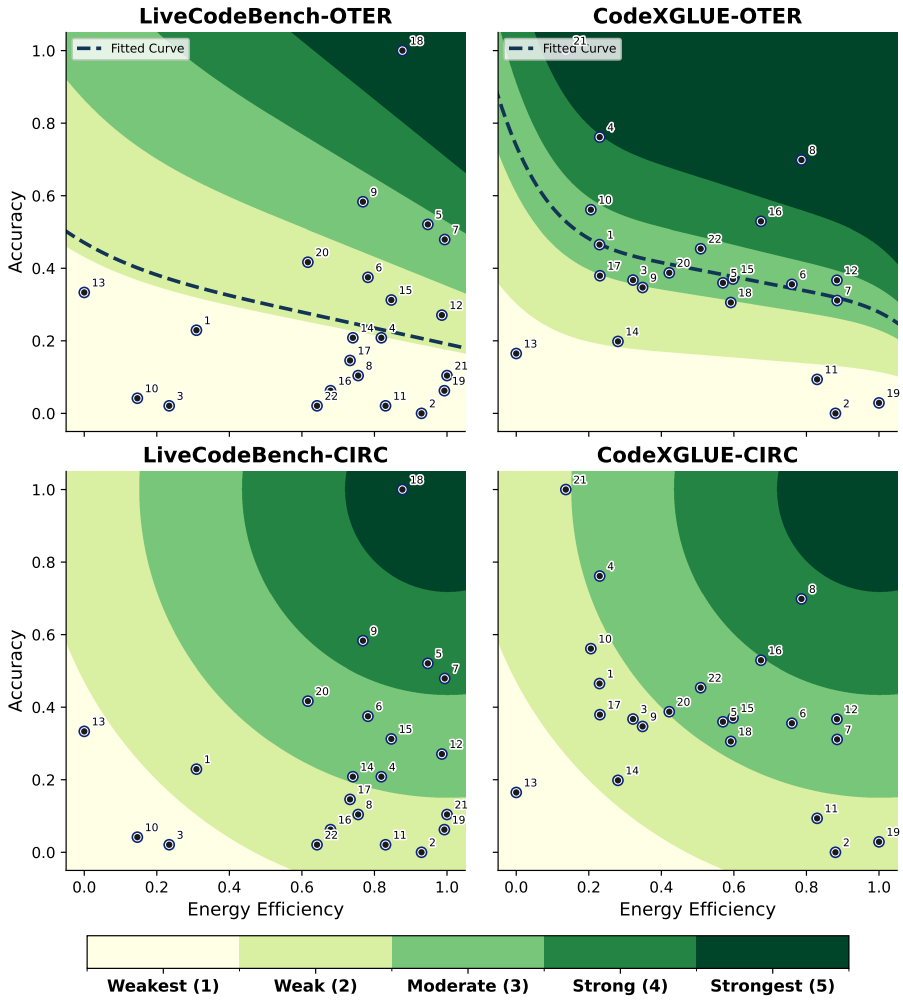

Fig. 7. Unified energy-accuracy model rating. Numbers in the plot show model IDs.

utilization. To validate that ratings reflect genuine trade-offs rather than systematic size bias, we conduct Kruskal-Wallis tests—a non-parametric method suitable for small samples without assuming normality [77, 93]. We group models by size (< 3B: $n = 6$, 3–7B: $n = 5$, ≥ 7B: $n = 11$) and test the null hypothesis that groups share the same rating distribution. Results fail to reject the null hypothesis ($p > 0.05$ for all: LCB-***CIRC*** $p = 0.92$, LCB-***OTER*** $p = 0.97$, CXG-***CIRC*** $p = 0.30$, CXG-***OTER*** $p = 0.40$), confirming no significant size-based bias.

Rating Methods Behavior Interpretation: We interpret how each method rates models based on their observed performance, shown in Figure 7. In the top-left plot, we observe ***OTER*** fits a curve to estimate the trend. It ranks over half the models in the weakest class. This results from the highly capable model M_{18}, which achieves perfect accuracy and near-optimal energy efficiency. Despite the presence of outliers and models exhibiting high energy efficiency but low accuracy, ***OTER*** does not favor them, demonstrating robustness.

The top-right plot shows that ***OTER*** rates the majority of models in the middle classes, as they reflect a balanced trade-off between energy efficiency and accuracy. The representation shows robustness at the highest energy efficiency, where moderate accuracy is expected, unlike M_2, M_{11}, and M_{19}. It also fairly rewards the most accurate model with relatively low energy efficiency (M_{21}), assigning it the highest score as it is considered as an exceptional outlier.

To statistically validate ***OTER***'s robustness to design choices, we conduct systematic sensitivity analyses on two critical aspects: hyperparameter selection and outlier removal. ***OTER***'s methodology involves explicit design choices, namely polynomial degree (5), MCD outlier threshold (95th percentile), LES quantile (75th), and ϵ (0.01) at $f(x = 1)$, that could introduce fragility. We test 358 hyperparameter configurations varying polynomial degree $\in \{3, 4, 5, 6, 7\}$, MCD percentile $\in \{90, 95, 97.5\}$, LES quantile $\in \{65, 70, 75, 80\}$, and $\epsilon \in \{0.001, 0.01, 0.1\}$ computing Spearman correlations between each configuration's ratings and our baseline. The ratings are highly stable, showing mean $\rho = 0.982$ (min $\rho = 0.922$), with all 358 configurations ($5 \times 3 \times 4 \times 3 = 180$ across 2 benchmarks, excluding 2 baselines) achieving $\rho > 0.85$ and maintaining 96.1% average rating stability (min 81.8%). All correlations are statistically significant ($p < 0.001$). To assess the critical role of outlier removal, we compare ratings with and without MCD-based coordinate filtering. We find that filtering is mathematically essential: without filtering, the regression curve is severely distorted by high-leverage points, dropping the Spearman correlation to $\rho = 0.628$ on the CXG dataset and reducing overall average stability to 72.7%. This confirms that MCD-based outlier removal therefore acts as a necessary stabilization mechanism that improves robustness to architectural breakthroughs without introducing systematic size bias.

The bottom left plot demonstrates the ***CIRC*** approach, which rates models regarding the circle that they fall into. Since this is a Euclidean-based ranking, both objectives are considered equally. This plot is predominantly dense in the weak class, as most models perform well on only one objective. The absolute distances along each curve allow fair comparison among models within the class. The bottom right plot also exhibits similar behavior. As a static scoring approach, ***CIRC*** leaves the very strong class empty, highlighting room for model improvement.

Having validated ***OTER***'s hyperparameter robustness and confirmed size-agnostic fairness for both methods, we further assess rating stability and fairness under two practical deployment scenarios: whether individual models disproportionately influence the overall ratings, and whether measurement noise in energy–accuracy inputs destabilizes rankings. We conduct two targeted tests addressing these concerns. First, leave-one-out (LOO) analysis—removing each model sequentially—assesses dependency on individual data points. Results demonstrate high stability: ***CIRC*** achieves mean rating drift of 0.024 (LCB) and 0.022 (CXG) with Kendall-$\tau > 0.98$, while ***OTER*** maintains 0.037 (LCB) and 0.067 (CXG) with Kendall-$\tau > 0.96$, confirming that ratings remain consistent regardless of which model is excluded. Second, we test robustness to measurement noise by perturbing inputs with uniform noise ($\epsilon = \pm 5\%$) across 20 trials. Both methods are resilient: ***CIRC*** shows mean rating changes of 0.032 (LCB) and 0.059 (CXG) with worst-case shifts of one rating class, and ***OTER*** 0.068 (LCB) and 0.209 (CXG), with worst-case shifts of one and two rating class respectively. These results confirm both methods yield stable ratings robust to model influence and measurement uncertainty.

Difference in *OTER* and *CIRC* on LiveCodeBench:

We compare how ***OTER*** behaves differently from ***CIRC***. Table 1 and the first column of Figure 7 show that the score orders under both methods are similar, with most models differing by only one class. ***CIRC*** is more lenient than ***OTER***, assigning higher ranks to models with at least one high objective. Both methods rank fairly with equal orders and differ only in the middle classes. Because of the discretization, many models sit close to boundaries, so a minor change in one value can move their class, which also causes slight variance in both methods.

Difference in *OTER* and *CIRC* on CodeXGLUE:

For CodeXGLUE, the CXG ratings in Table 1 show more conflicts, which arise from the relations that ***OTER*** captures. In the right column of Figure 7, granite-8b-code-base-4k (M_{21}) is rated "Strongest" by ***OTER***, which reflects the difficulty of reaching high accuracy, while it is "Weak" under ***CIRC***, which reflects its low energy efficiency. As on LiveCodeBench, ***OTER*** on CodeXGLUE does not favor the most energy-efficient models. Most models cluster at moderate accuracy (0.3–0.6) while spanning the full energy efficiency range, which shows that high efficiency is easier to reach than high accuracy, since few models reach high accuracy at any efficiency level. Both methods behave closely, with some exceptions that highlight their distinct behavior depending on the perspective taken.

Benchmark Effects on Ratings:

In the first row of Figure 7, which shows ***OTER*** results, scores are generally lower on LiveCodeBench. This reflects the higher complexity of generating structured, grammar-based text that also passes unit tests, compared to open-ended text generation under a less rigid evaluation metric. ***OTER*** captures this, because it stays trend-normalized per task and assigns a higher expected accuracy to CodeXGLUE without trivially favoring the easier task. The second row in the Figure 7 shows model ratings using ***CIRC***. While models on CodeXGLUE have higher average accuracy, reflecting the task's relative ease, their low energy efficiency places most models in the second and third classes.

Model ratings can also be compared across the LCB and CXG columns of Table 1. Variances are more pronounced under ***OTER***, where the fitted curve produces model-specific scoring, and we highlight the largest differences. The first, deepseek-coder-1.3b-base (M_8), performs well on CodeXGLUE with 0.7 accuracy and 0.79 energy efficiency, giving scores of 4 and 5. On LiveCodeBench its accuracy drops to 0.1 despite a reasonable energy efficiency of 0.76, giving scores of 2 (***CIRC***) and 1 (***OTER***). This is expected, because ***OTER*** treats high energy efficiency as easier to reach than high accuracy, so this small model struggles on the more demanding LiveCodeBench and shows a large score gap across the two tasks. A second example, Seed-Coder-8B-Instruct (M_{18}), shows high energy efficiency on both tasks, so the score gap is driven by accuracy. It scores a perfect 5 on LiveCodeBench but only 3 (***CIRC***) and 2 (***OTER***) on CodeXGLUE, which suggests it excels at code generation but is less capable at code comprehension.

Summary

- Models generally score higher on CodeXGLUE because the task is easier.
- Because accuracy shows low variance across most models, ***OTER*** treats high accuracy as harder to reach and rewards it more.
- ***CIRC*** is a reliable, deterministic method, while ***OTER*** is trend-aware and, with outlier removal, robust to individual extreme points.
- Trade-offs in ***CIRC*** are static, while trade-offs in ***OTER*** depend on a model's position.
- Final scores can vary widely between the two tasks, which depends on architecture and the pretraining phase, specifically how much emphasis fell on code comprehension versus generation.

5.2 RQ2: Rating Distribution Shift

In a real-world setting, new models get released every day. So any good rating mechanism needs to be flexible enough to ingest this continuous flow of new models while maintaining the original set in a just manner. However, when these new models are added, they can shift existing model ratings, warranting investigation into whether original ratings stay stable, whether any shifts are justified, and whether the newcomers receive fair scores. To investigate this, we

extend the base cohort with 15 new models, of which 4 are incompatible with the current experimental setup, leaving us with 11 additional models. The new models are selected by the same protocol used for the original set of models to ensure a realistic comparison, just 6 months apart. This raises the total to 33, and we re-rate the full set to answer this research question. Table 2 and Figure 8 report the updated ratings.

Why a shift is expected, and what it means.: Both methods are relative by construction, because they anchor the ends of the bounded 1–5 scale to the best and worst observed behavior through min-max normalization. Both objectives, energy and accuracy, are normalized with the same preprocessing step, a standard practice that ensures comparability between objectives and is widely recommended in MCDM and TOPSIS-style ranking methods [21, 71, 73, 102]. This normalization lets ***CIRC*** define its five concentric regions within a bounded $[0, 1]$ range and lets ***OTER*** map both axes onto a common scale, which gives a more stable curve fit.

A direct consequence of this relative design is that any new model that breaks the current minimum or maximum shifts the normalized coordinates for the whole cohort. For ***OTER***, new data can also come with a refit of the expectation curve to capture the updated trend. This recalibration is essential, because the open-weight ecosystem keeps pushing toward greater capability at lower cost, so a static benchmark would reward obsolete trade-offs. Refitting ensures that every model, old and new, is evaluated against the current frontier rather than an outdated expectation. If an existing model's rating shifts under both methods, this does not mean its absolute performance degraded, only that the baseline moved. The measure of stability in an evolving benchmark is therefore not whether individual ratings stay frozen, but whether the relative *ordering* of the original models is preserved.

The original ordering is preserved.: Comparing the 22 original models before and after the expansion, the ordering is highly stable: Kendall-$\tau = 1.00$ for ***CIRC*** on both benchmarks, and $\tau = 0.97$ (LiveCodeBench) and 0.87 (CodeXGLUE) for ***OTER***. ***CIRC*** leaves every original rating untouched, while ***OTER*** reorders only mildly, with 3 models shifting on LiveCodeBench, where two of them moved up by one class, and one of them improved by two classes, and 5 shifting on CodeXGLUE, each by a single class and always upward, with no demotions. The lower ***OTER*** agreement on CodeXGLUE reflects its larger count of these one-class adjustments rather than any genuine reshuffling. Because the newcomers fall inside the existing distribution rather than redrawing its extremes, we observe no significant change in the scoring methods.

Where the new models fall.: On LiveCodeBench the new models cluster at low-to-moderate accuracy with high energy efficiency, settling into the already-dense lower-right of the plane; they neither extend the observed range nor disturb min–max, so both methods' frames of reference remain intact. On CodeXGLUE the range does move, driven by a single newcomer: M_{33} returns only one token per sample, evidently failing to parse the prompt template and finishing almost immediately, which makes it simultaneously the least accurate model and, by consuming roughly an order of magnitude less energy than any other, the most efficient. This nudges the normalized coordinates (M_2's accuracy by 0.05, M_{19}'s efficiency from 1.00 to 0.95), but by too little to cross any rating boundary.

CIRC*: invariant by construction.:** Because ***CIRC scores a model by its absolute Euclidean distance to the ideal point $(1, 1)$, its concentric boundaries depend only on the normalized range, never on how many models populate it. On LiveCodeBench the observed extremes of both axes are unchanged, so the five regions, and all original ratings are identical. On CodeXGLUE the small range shift introduced by M_{33} is too slight to push any model across a boundary, so every original ***CIRC*** rating is retained, and the newcomers slot in beside existing models of comparable profile and inherit comparable scores. ***CIRC*** thus behaves as a fixed, count-independent method: any model can be dropped onto

the plane and scored by the region it lands in, the existing ratings do not vary, and a recomputation is triggered only when the range itself moves.

OTER* on LiveCodeBench, the expectation curve steepens.:** The new models concentrate heavily in the high-efficiency, low-accuracy corner, intensifying the dominant pattern that efficiency is easy to attain while accuracy is hard. Compared to the previous case, *i.e.,* ***OTER on 22 models (Figure 7), where at zero efficiency ($f(x = 0)$) the curve expected average accuracy—***OTER*** now responds by steepening its curve in Figure 8. Because low-efficiency models are now scarce outliers, the framework adapts to expect peak accuracy at zero efficiency. This steepness is also supported by the structural constraint that expected accuracy cannot exceed 1, causing the curve to down-weight those few low-efficiency outliers and focus on the dense population spanning the 0.5 to 1 efficiency range.

Two distinct regional consequences emerge from this shift. First, strictness at the lower efficiency spectrum widens the rating-1 region. Because the vast majority of models are now highly efficient, the framework penalizes low efficiency severely; at the extreme, even a highly accurate model would now receive a low rating if its efficiency is poor. Second, at the high-efficiency spectrum, the curve relaxes compared to the stricter baseline in Figure 7. Acknowledging that strong accuracy is exceptionally difficult to achieve alongside high efficiency, ***OTER*** lowers its expectations here, shifting rating regions like class 4 rightward and downward to reward efficient models more generously.

This adaptive behavior directly explains the shifts in individual model evaluations. The Qwen family variants (M_5, M_7, and M_{12}) saw their ratings improve by 1, 2, and 1 classes, respectively. This reassessment is fair: they are the only models capable of sustaining respectable accuracy at such high levels of efficiency, making them stand out against the broader trend. While M_7's climb of two classes is a discretization effect caused by its position on a regional boundary rather than a massive score divergence, its upgrade remains justified. Ultimately, these shifts demonstrate how flexibly and intelligently ***OTER*** captures evolving model trends.

OTER* on CodeXGLUE, the expectation curve linearizes.:** Analyzing ***OTER for the CodeXGLUE benchmark reveals a distinct shift from the baseline in Figure 7. The new models concentrate primarily around moderate accuracy while spanning the entire energy efficiency spectrum, flattening the curve into a nearly linear fit once outliers are removed. Despite this geometric change, the overall area occupied by each rating class remains stable. The curve essentially concedes that achieving high accuracy is challenging even at the lowest efficiency levels, prompting ***OTER*** to become slightly more lenient.

This shifting baseline has two primary consequences. First, because the fitted curve demonstrates a mild slope, the regions reaffirm that accuracy remains the scarce, highly rewarded resource for code summarization. Second, this leniency directly impacts individual model placements: five boundary-adjacent models (M_4, M_9, M_{10}, M_{17}, and M_{18}) each move up by one rating class due to this slight shift in accuracy expectations, whereas they had previously been ranked one class lower by the effects of discrete class boundaries. The remaining original models maintain their baseline ratings. This adaptation ensures fairness for the newcomers; under the stricter expectations of the original curve, these average-accuracy models would have been penalized with a lower class. Refitting thus preserves an equitable comparison across both the old and newly introduced models.

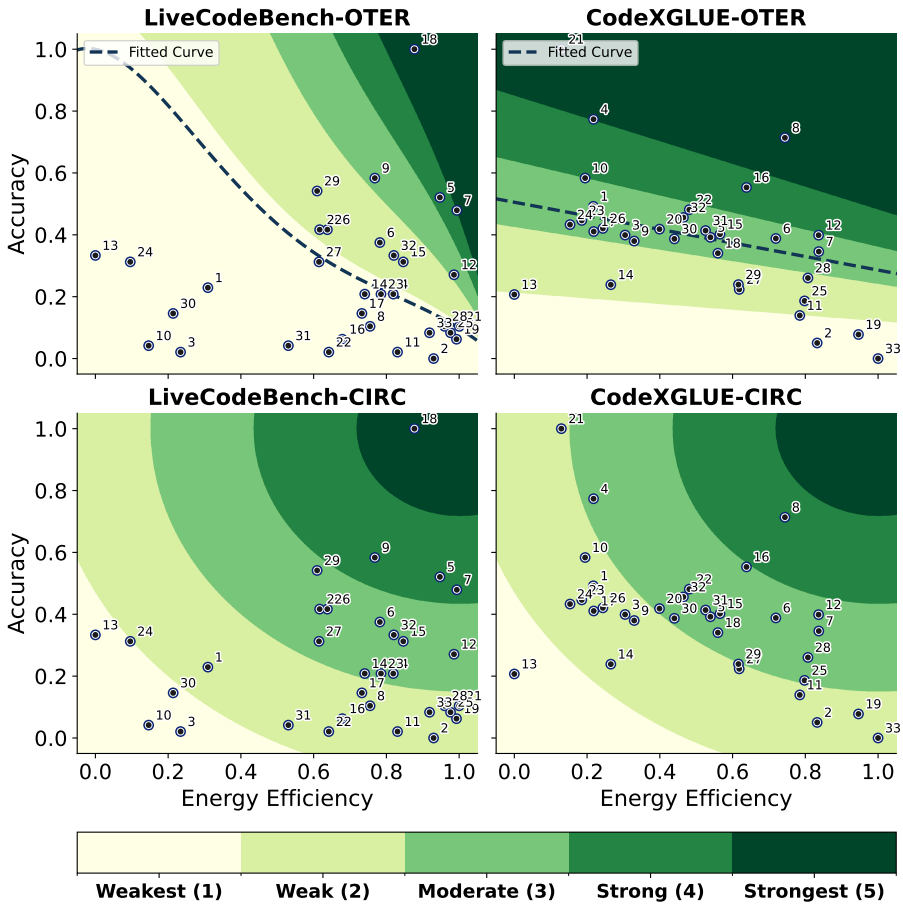

Fig. 8. Unified energy-accuracy model rating; numbers in the plot show models' IDs.

Summary

- Growing the cohort from 22 to 33 preserves the ranking: ***CIRC*** is unchanged (Kendall-τ = 1.00) and ***OTER*** reorders only mildly ($\tau \geq 0.87$), always by a single class and upward, with no demotions.
- A rating shift for an existing model means the reference distribution moved, not that the model changed; the model is simply re-measured against current set.
- ***CIRC*** is count-independent—its absolute distance-to-ideal scoring changes only when the normalized range moves, and then predictably—so newcomers are scored by the same distance criteria as the original set.
- ***OTER*** refits to the new trend by design; its steepened (LiveCodeBench) and linearized (CodeXGLUE) curves encode genuine ecosystem updates, and refitting protects newcomers from an outdated expectation.

ID	Model	LCB		CXG		LCB Rating		CXG Rating	
		A	*E*	*A*	*E*	CIRC	OTER	CIRC	OTER
M_1	deepseek-coder-6.7b-base	0.23	0.31	0.49	0.22	2	1	2	3
M_2	starcoderbase-1b	0	0.93	0.05	0.83	2	1	2	1
M_3	starcoder2-3b	0.02	0.23	0.4	0.3	1	1	2	3
M_4	CodeLlama-7b-Instruct-hf	0.21	0.82	0.77	0.22	3	1	3	5
M_5	Qwen2.5-Coder-3B-Instruct	0.52	0.95	0.39	0.54	4	4	3	3
M_6	Qwen2.5-Coder-7B-Instruct	0.38	0.78	0.39	0.72	3	2	3	3
M_7	Qwen2.5-Coder-1.5B	0.48	0.99	0.35	0.84	4	5	3	3
M_8	deepseek-coder-1.3b-base	0.1	0.76	0.71	0.74	2	1	4	5
M_9	deepseek-coder-7b-instruct-v1.5	0.58	0.77	0.38	0.33	4	3	2	3
M_{10}	starcoder2-7b	0.04	0.15	0.58	0.19	1	1	2	4
M_{11}	codegen-350M-mono	0.02	0.83	0.14	0.78	2	1	2	1
M_{12}	Qwen2.5-Coder-0.5B	0.27	0.99	0.4	0.84	3	3	3	3
M_{13}	Yi-Coder-9B	0.33	0	0.21	0	1	1	1	1
M_{14}	rombos_Replete-Coder-Llama3-8B	0.21	0.74	0.24	0.27	3	1	2	2
M_{15}	speechless-code-mistral-7b-v1.0	0.31	0.85	0.4	0.57	3	2	3	3
M_{16}	stable-code-3b	0.06	0.68	0.55	0.64	2	1	3	4
M_{17}	CodeLlama-7b-Python-hf	0.15	0.73	0.41	0.22	2	1	2	3
M_{18}	Seed-Coder-8B-Instruct	1	0.88	0.34	0.56	5	5	3	3
M_{19}	CodeQwen1.5-7B-Chat	0.06	0.99	0.08	0.95	2	1	2	1
M_{20}	Magicoder-S-DS-6.7B	0.42	0.62	0.42	0.4	3	2	3	3
M_{21}	granite-8b-code-base-4k	0.1	1	1	0.13	2	1	2	5
M_{22}	codegen-2B-mono	0.02	0.64	0.48	0.48	2	1	3	3
M_{23}	**speechless-zephyr-code-functionary-7b**	0.21	0.79	0.45	0.19	3	1	2	3
M_{24}	**OpenCoder-8B-Instruct**	0.31	0.1	0.43	0.15	1	1	2	3
M_{25}	**llama-3.2-1b-code-instruct**	0.08	0.98	0.19	0.8	2	1	3	2
M_{26}	**OpenCodeInterpreter-DS-6.7B**	0.42	0.64	0.42	0.24	3	2	2	3
M_{27}	**mini-coder-1.7b**	0.31	0.61	0.22	0.62	3	1	2	2
M_{28}	**codegemma-2b**	0.1	0.96	0.26	0.81	2	1	3	2
M_{29}	**Qwen3-1.7B-Sushi-Coder**	0.54	0.61	0.24	0.62	3	2	2	2
M_{30}	**CodeNinja-1.0-OpenChat-7B**	0.15	0.21	0.39	0.44	1	1	3	3
M_{31}	**qwen2.5-coder-3b-claude_opus_4.6**	0.04	0.53	0.41	0.53	2	1	3	3
M_{32}	**codegemma-7b-it**	0.33	0.82	0.46	0.46	3	2	3	3
M_{33}	**prism-coder-7b**	0.08	0.92	0	1	2	1	2	1

Table 2. Summary of model performance on LiveCodeBench (LCB) and CodeXGLUE (CXG): normalized accuracy (*A*), normalized efficiency (*E*) and unified ratings (CIRC and OTER) for each benchmark. TODO ▶ *javad, would it make sense to highlight the newq once here by slight different shape or Bold to emphasize them ?* ◀

TODO ▶ *@saurabh, I've bolded them, but it didn't appear well. we can say in caption that the new ones are from M23 to M33, or if you have any other styling option, let me know.* ◀ TODO ▶ *Saurav: Sure I will update it, I feel we need a plot/figure to visualize it better, how the rating distribution differs, like two histograms maybe or something better, to show before and after of each model shift. As a reader, I would want to see how the 5 clusters are shifting essentially, and if possible we can show adding one model at a time this "entropy" basicly evolving ratings, just for the figure, starting from 1 to all 11, wiuld be a nice figure to understand the behaviour* ◀

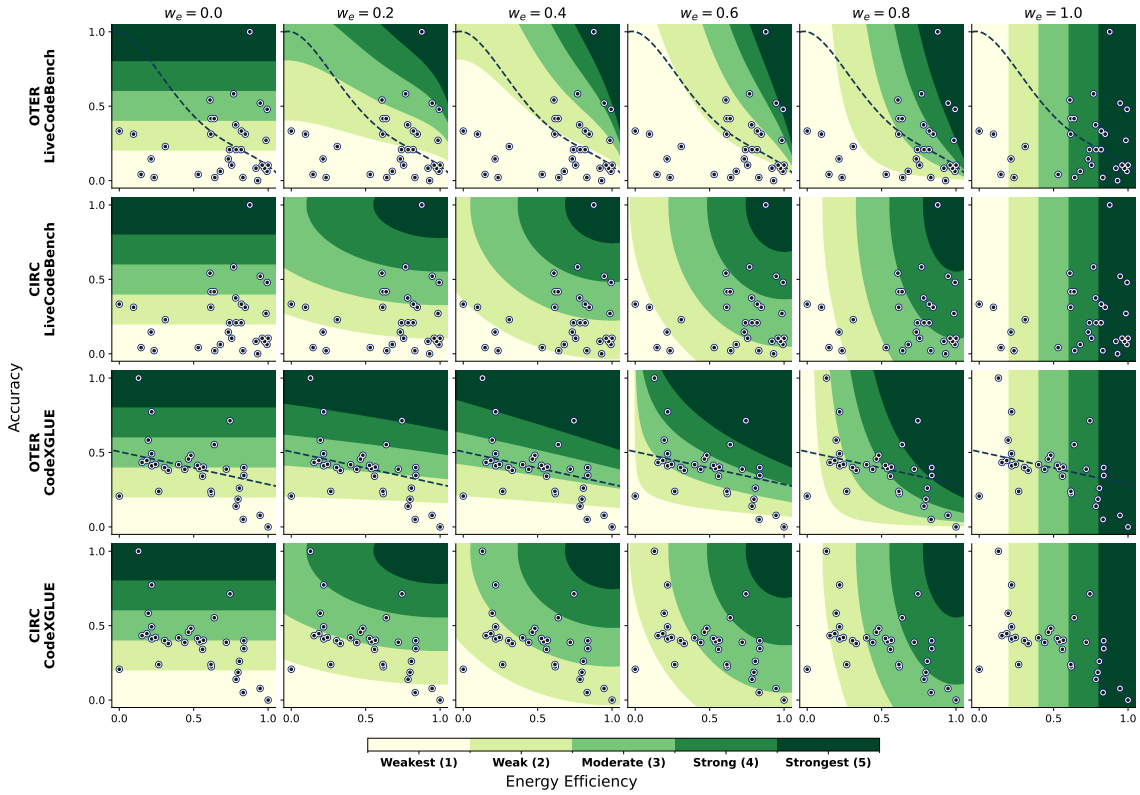

Fig. 9. Evolution of the energy–accuracy rating regions as the energy priority weight w_e increases from 0 (accuracy-only) to 1 (energy-only). Rows show ***OTER*** and ***CIRC*** on each benchmark; points are the 33 models.

5.3 RQ3: Priority Elasticity

Equal weighting is a principled default, but real deployments impose asymmetric priorities, since an edge coding assistant may care foremost about energy while a mission-critical code review agent may care about accuracy. We therefore sweep the priority weights (w_e for energy, $w_a = 1 - w_e$ for accuracy) and verify that the weighted formulations of ***CIRC*** and ***OTER*** (Equations 11 and 17) steer the ratings as intended while preserving their mathematical integrity. Figure 9 shows how the rating regions evolve across $w_e \in \{0, 0.2, \ldots, 1\}$, and Figures 10 and 11 quantify continuity and sensitivity.

The rating field deforms predictably.: At $w_e = 0$, only accuracy matters, so the boundaries are horizontal bands where sliding a model along the efficiency axis never changes its rating. As w_e grows to 1, these boundaries progressively tilt into vertical bands governed purely by efficiency. This shift penalizes low-efficiency models and rewards high-efficiency ones, causing models that scored highest under balanced weights to drop once their energy penalties become too severe.

CIRC* boundaries change predictably on both tasks.:** For ***CIRC the concentric circles deform into ellipses stretched along the de-emphasized axis: horizontally when accuracy leads, so a small accuracy gain crosses a class while efficiency is ignored, and vertically when energy leads, so the reverse holds. Near the balanced midpoint the boundaries resembles the baseline geometry; toward either extreme the regions equalize in area as the score collapses onto a single linear axis.

OTER*'s weight-dependent boundary dynamics on LiveCodeBench and CodeXGLUE.:** On LiveCodeBench (first row), near the balanced center ($w_e = 0.5$), the boundaries mirror the baseline curve and reflect the complexity of $y \times \frac{1}{f(x)}$, whereas approaching the minimum or maximum weight extremes forces the boundary areas within each region to become approximately equal. A similar progression occurs on CodeXGLUE (third row), where the near-linear baseline curve clearly exposes ***OTER's two piecewise weighting branches. For $w_e < 0.5$, the score reduces to $y/f(x)^{2w_e}$; because $f(x)$ is nearly linear and the exponent is below 1, the boundaries stay almost linear. For $w_e > 0.5$, the score becomes $x^{2w_e-1} \cdot f(x)^{-2w_a} \cdot y^{2w_a}$, where the $x^{2w_e-1} \cdot f(x)^{-2w_a}$ factor explicitly bends the boundaries away from linearity.

Both methods collapse to the same single-axis rating at the extremes. At $w_e = 0$ and $w_e = 1$ the weighted score reduces to a single linear coordinate, so the two otherwise-distinct methods must produce the same uniform binning. They do: ***CIRC*** and ***OTER*** assign identical ratings to all 33 models at both extremes on both benchmarks. This confirms that the piecewise ***OTER*** formulation degenerates cleanly to pure accuracy or pure efficiency with equal-width intervals, meeting single-objective demands, and that the two methods agree wherever a single correct answer exists.

Reweighting moves the ranking smoothly, not abruptly.: TODO ▶ *we need some text here for explaining the justification of why this is important, the continuity aspect, also need to refine this para a bit to make it less dense, like they will understand "what" is written but not get "why" exactly unless they read two three times* ◀ TODO ▶ *@saurav, could you check if this text is enough or i need to add more?* ◀ A good rating mechanism should also exhibit smooth and interpretable ranking transitions with respect to neighboring weight settings. If two ranking results corresponding to weight values that differ by only 0.05 produce substantially different rankings, it suggests that the mechanism is overly sensitive and introduces uncontrolled jumps, making the resulting changes difficult to interpret. Therefore, it is important to verify that ranking shifts remain within a reasonable range as the weights vary. To assess the absence of such discontinuities, we sweep w_e in increments of 0.05 and measure the Spearman correlation between rankings obtained from consecutive weight settings (Figure 10). The adjacent-step correlation stays high throughout (mean 0.93, never below 0.76 across both methods and benchmarks), so no weight increment reshuffles the order sharply—including across $w_a = w_e = 0.5$, where ***OTER*** switches piecewise branches, which confirms that the seam is continuous. Correlation against the balanced baseline ($w_e = 0.5$) peaks at 1 and decays monotonically toward both extremes, showing that the weights successfully re-order models rather than merely rescaling them. The decay is steepest for ***OTER*** on CodeXGLUE, reaching $\rho = -0.42$ at $w_e = 1$: because that task treats efficiency as easy to attain (Section 5.2), an efficiency-only ranking nearly inverts the balanced one. Moreover, since ***CIRC*** is a static geometric formula, it decays almost symmetrically and predictably.

Priorities steer the score in the intended direction.: We track three fixed archetypes, energy-dominant ($\mathit{eff} = 0.9$, $\mathit{acc} = 0.1$), accuracy-dominant (0.1, 0.9), and balanced (0.5, 0.5), as w_a sweeps from 0 to 1, expressing each score as its percentile ranking within a dense synthetic field of operating points (Figure 11). We expect the energy-dominant point to decrease monotonically as w_a increases, the accuracy-dominant point to increase monotonically, and the balanced point to stay mid-range. We observe exactly this. The accuracy-dominant archetype rises and the energy-dominant one falls (Spearman $\rho = +1.00$ and -1.00 in every panel), while the balanced archetype holds mid-scale. The two dominant curves cross where priority tips. For ***CIRC*** the crossing sits at $w_a = 0.5$, as its geometric symmetry dictates. For ***OTER*** it occurs earlier, because ***OTER*** treats accuracy as the harder objective, so at equal weights an accuracy-dominant point already outranks an energy-dominant one and the two curves meet before the midpoint. The shape of these curves follows the fitted baseline. On LiveCodeBench the curve has a non-trivial slope, so the transition is smooth and the two archetypes stay relatively close near balance, and an efficiency of 0.9 still buys a respectable balanced score. On CodeXGLUE the energy-dominant archetype drops abruptly from $w_a = 0$ to 0.5, because under balanced weights this

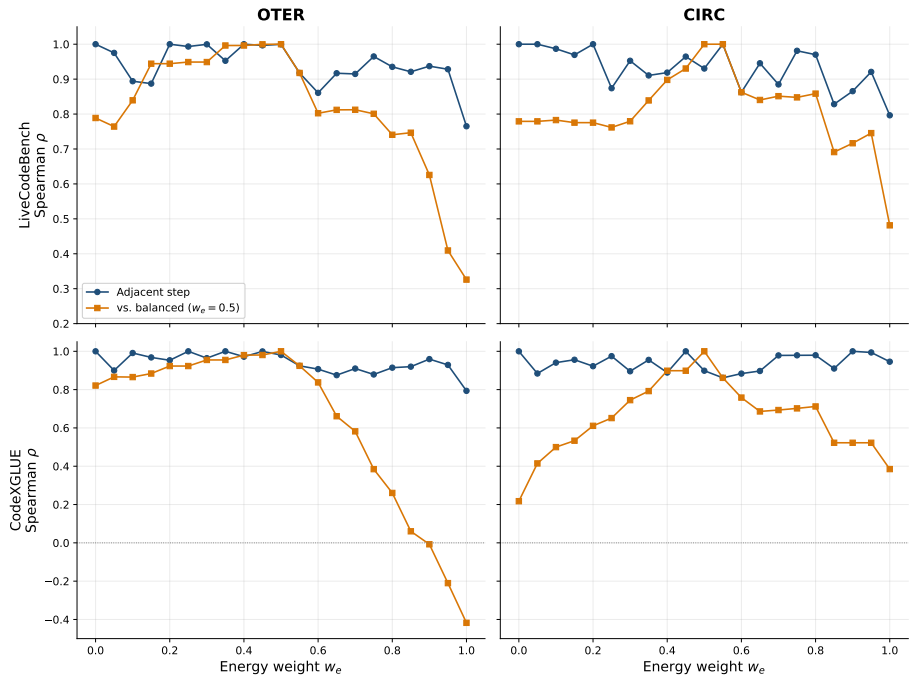

Fig. 10. Spearman rank correlation of the model ordering under a weight sweep, comparing each step against the previous one (local continuity) and against the balanced w_e=0.5 baseline (cumulative reordering). TODO ▶ *we can increase the label/legend font sizes, also the scale numbers*◀

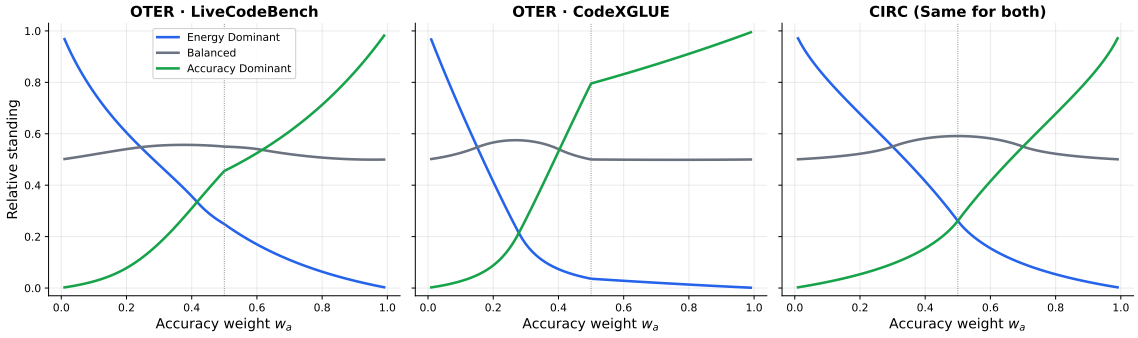

Fig. 11. Relative standing of dominant-objective archetypes (*eff*=0.9, *acc*=0.1 or vice versa) and a balanced one (0.5, 0.5) as the accuracy weight w_a increases, scored against a dense synthetic field. ***OTER*** depends on the per-task curve, while ***CIRC*** is geometric and task-independent.

task already scores efficient-but-inaccurate models low (Section 5.2), so the archetype falls from best to near-worst over that interval and then flattens at the lowest achievable score. From w_a = 0.5 to 1 the branch swaps to accuracy control, and because the baseline curve is near-linear the accuracy-dominant archetype improves almost linearly. As a final check, reweighting leaves robustness intact, since under ±5% input noise (200 trials, with ***OTER*** refit on each perturbed

sample) the mean rating drift stays small at every priority setting, including accuracy-first ($w_a = 0.9$) and energy-first ($w_e = 0.9$) (***CIRC*** ≤ 0.18 of a class, ***OTER*** ≤ 0.24), so the extreme accuracy- and energy-first profiles hold almost the same stability as the balanced weight.

Summary

- Weighted ***CIRC*** and ***OTER*** turn the fixed energy–accuracy rating into a tunable one without sacrificing interpretability: the regions deform continuously from accuracy-only to energy-only as priority shifts.
- At both single-objective extremes the two methods provably coincide, rating all 33 models identically with uniform bins, and adjacent-weight rankings never jump ($\rho \geq 0.76$ at every step).
- Priorities steer ratings monotonically ($\rho = \pm 1.00$), crossing at $w_a = 0.5$ for ***CIRC*** and earlier for ***OTER***, which intrinsically treats accuracy as the harder objective.
- Ratings stay noise-robust at every weighting (drift ≤ 0.24 for ***OTER***, ≤ 0.18 for ***CIRC***), so practitioners can dial priorities without losing stability.

5.4 RQ4: Scaling Laws Efficiencies

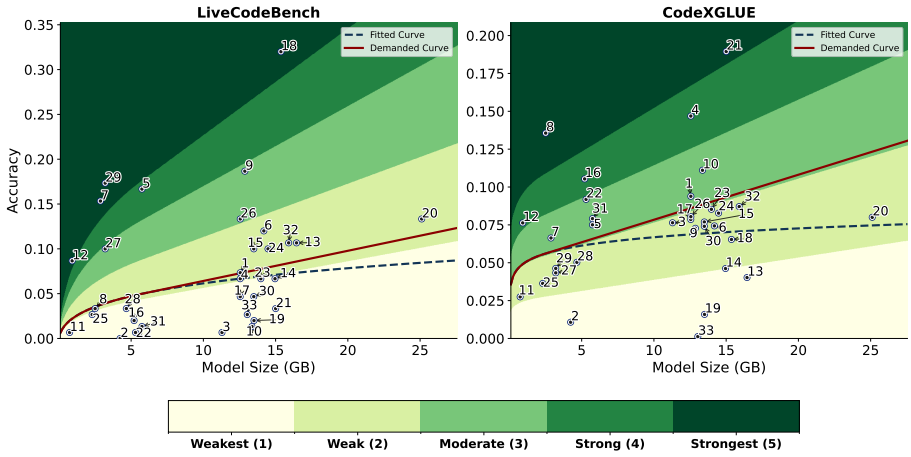

Fig. 12. Unified size–accuracy model rating. The dashed line is the fitted scaling trend $\hat{f}_{acc}$ and the solid line the size-adjusted demanded curve $\mathcal{G}$; rating regions are defined by each model's position relative to $\mathcal{G}$. Numbers in the plot show model IDs.

To answer RQ4, we examine how efficiently each *CLM* converts its physical footprint into task accuracy (*Performance Efficiency*, ψ) and into energy savings (*Structural Efficiency*, τ), using the size–accuracy and size–energy scaling curves introduced in Section 4.4. Table 3 reports the size, raw accuracy, raw energy, and the resulting ψ and τ ratings for all models on both benchmarks, while Figures 12 and 13 visualize the corresponding rating regions. Unlike the normalized values in Tables 1 and 2, the accuracy and energy columns here are raw, since the scaling curves operate on raw accuracy, raw energy, and raw size.

Why size belongs in the evaluation.: Figures 12 and 13 show why size matters: models of nearly identical footprint differ markedly in accuracy and energy. In the size–accuracy view, the fitted curve, which represents the scaling law, encodes the accuracy expected at each size. Models *above* it extract more capability than their scale predicts, which

ID	Model	Size (GB)	LCB		CXG		LCB Rate		CXG Rate	
			A	E	A	E	ψ	τ	ψ	τ
M_1	deepseek-coder-6.7b-base	12.56	0.07	1.25e+06	0.09	1.23e+06	2	2	3	1
M_2	starcoderbase-1b	4.24	0	1.95e+05	0.01	2.68e+05	1	5	1	4
M_3	starcoder2-3b	11.29	0.01	1.38e+06	0.08	1.09e+06	1	1	2	2
M_4	CodeLlama-7b-Instruct-hf	12.55	0.07	3.84e+05	0.15	1.23e+06	2	4	4	1
M_5	Qwen2.5-Coder-3B-Instruct	5.75	0.17	1.65e+05	0.08	7.25e+05	4	5	3	2
M_6	Qwen2.5-Coder-7B-Instruct	14.19	0.12	4.47e+05	0.07	4.45e+05	2	4	2	4
M_7	Qwen2.5-Coder-1.5B	2.88	0.15	8.57e+04	0.07	2.61e+05	5	5	3	4
M_8	deepseek-coder-1.3b-base	2.51	0.03	4.93e+05	0.14	4.06e+05	2	3	5	3
M_9	deepseek-coder-7b-instruct-v1.5	12.87	0.19	4.70e+05	0.07	1.05e+06	3	4	2	2
M_{10}	starcoder2-7b	13.36	0.01	1.53e+06	0.11	1.26e+06	1	1	3	1
M_{11}	codegen-350M-mono	0.74	0.01	3.64e+05	0.03	3.43e+05	1	2	2	2
M_{12}	Qwen2.5-Coder-0.5B	0.92	0.09	9.90e+04	0.08	2.62e+05	5	5	4	3
M_{13}	Yi-Coder-9B	16.45	0.11	1.78e+06	0.04	1.57e+06	2	1	1	1
M_{14}	rombos_Replete-Coder-Llama3-8B	14.96	0.07	5.17e+05	0.05	1.15e+06	1	4	2	2
M_{15}	speechless-code-mistral-7b-v1.0	13.49	0.1	3.37e+05	0.08	6.84e+05	2	5	2	3
M_{16}	stable-code-3b	5.21	0.02	6.23e+05	0.11	5.71e+05	1	3	4	3
M_{17}	CodeLlama-7b-Python-hf	12.55	0.05	5.31e+05	0.08	1.23e+06	1	4	2	1
M_{18}	Seed-Coder-8B-Instruct	15.37	0.32	2.85e+05	0.07	6.93e+05	5	5	2	3
M_{19}	CodeQwen1.5-7B-Chat	13.5	0.02	8.73e+04	0.02	9.13e+04	1	5	1	5
M_{20}	Magicoder-S-DS-6.7B	25.11	0.13	7.29e+05	0.08	9.44e+05	2	4	2	3
M_{21}	granite-8b-code-base-4k	15	0.03	7.49e+04	0.19	1.36e+06	1	5	5	1
M_{22}	codegen-2B-mono	5.3	0.01	6.86e+05	0.09	8.16e+05	1	3	3	2
M_{23}	speechless-zephyr-code-7b	13.98	0.07	4.41e+05	0.09	1.28e+06	1	4	2	1
M_{24}	OpenCoder-8B-Instruct	14.48	0.1	1.62e+06	0.08	1.33e+06	2	1	2	1
M_{25}	llama-3.2-1b-code-instruct	2.3	0.03	1.15e+05	0.04	3.22e+05	1	5	2	4
M_{26}	OpenCodeInterpreter-DS-6.7B	12.56	0.13	6.93e+05	0.08	1.19e+06	3	3	2	1
M_{27}	mini-coder-1.7b	3.2	0.1	7.32e+05	0.04	6.02e+05	3	2	2	2
M_{28}	codegemma-2b	4.67	0.03	1.43e+05	0.05	3.08e+05	1	5	2	4
M_{29}	Qwen3-1.7B-Sushi-Coder	3.2	0.17	7.41e+05	0.05	6.05e+05	5	2	2	2
M_{30}	CodeNinja-1.0-OpenChat-7B	13.49	0.05	1.42e+06	0.07	8.80e+05	1	1	2	3
M_{31}	qwen2.5-coder-3b-claude_opus	5.75	0.01	8.75e+05	0.08	7.46e+05	1	2	3	2
M_{32}	codegemma-7b-it	15.9	0.11	3.81e+05	0.09	8.41e+05	2	5	2	3
M_{33}	prism-coder-7b	13.04	0.03	2.14e+05	0	7.38e+03	1	5	1	5

Table 3. Summary of models' efficiency scores on LiveCodeBench (LCB) and CodeXGLUE (CXG): accuracy (A), energy consumption (E) and unified ratings, *i.e.*, $\psi(accuracy,\ size)$ and $\tau(energy,\ size)$, for each benchmark.

points to a stronger architecture, higher-quality training tokens, or better post-training, whereas models *below* it use their parameters inefficiently, which points to token starvation or architectural inefficiency and makes them candidates for pruning. The size–energy view is even more dispersed, because at a fixed size total energy varies by more than an order of magnitude. Around 13–16 GB, for example, LiveCodeBench energy ranges from 7.5×10^4 J for M_{21} to 1.78×10^6 J for M_{13}. Models below the energy curve are structurally efficient, either architecturally lean or disciplined about when to stop, while models above it over-consume, whether through heavier internal computation or runaway generation when they cannot converge on an answer. The two curves therefore turn size from a confound into a diagnostic baseline.

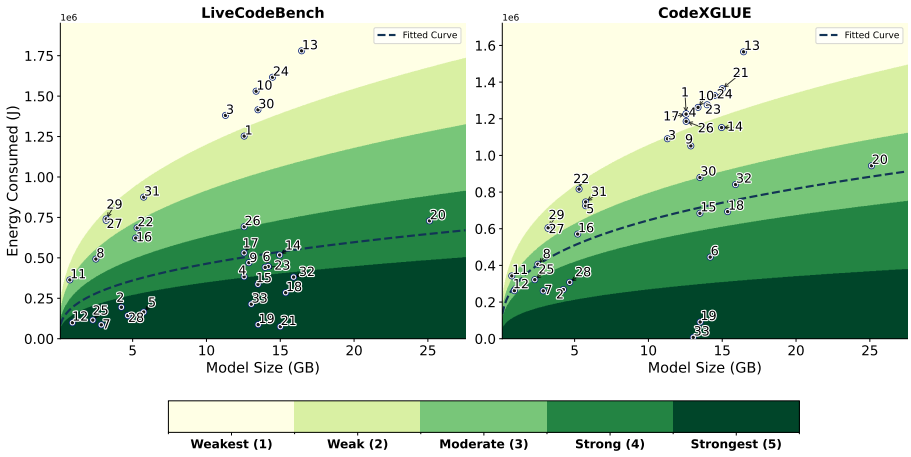

Fig. 13. Unified size–energy model rating. Numbers in the plot show model IDs.

Being above the scaling laws curve is necessary but not sufficient for Performance Efficiency.: A model above the fitted scaling curve converts size into accuracy more efficiently, which may reflect a stronger architecture, higher-quality training data, or better post-training, but the vertical deviation alone is not a fair basis for comparison. It fails to penalize models in the plateaued region of the scaling law, where large increases in size yield only marginal accuracy gains. As the Performance Efficiency plot (Figure 12) shows, the fitted size–accuracy trend is nearly flat, because even relatively small models already reach high accuracy. Within this regime, the scaling law permits large size increases for slight accuracy gains, so models with similar accuracy but very different sizes receive nearly identical scores under the $A/\hat{f}_{acc}(S)$ mechanism. The demanded curve addresses this by keeping the overall shape of the fitted law while enforcing higher accuracy requirements in the plateaued regime, so models that keep growing without matching accuracy gains are penalized even when they stay above the original curve. The demanded curve transitions to an approximately linear increase with slope μ, which establishes size-aware boundaries over the flattened region.

Because the demanded curve $\mathcal{G}$ lies on or above the fitted trend $\hat{f}_{acc}$, the size–accuracy plane splits into three regions with distinct meaning: *below* $\hat{f}_{acc}$ (under-performing for one's size), *between* $\hat{f}_{acc}$ and $\mathcal{G}$ (above the raw trend, but not by enough to justify the footprint), and *above* $\mathcal{G}$ (converting size into accuracy efficiently). The middle region is where the size adjustment acts, where mostly mid-to-large models are located here whose above-trend accuracy is too small to offset their scale. `Magicoder-S-DS-6.7B` (M_{20}, 25.1 GB) is the clearest case: on CodeXGLUE it lies above the fitted trend yet, at 0.08 accuracy, sits well below the demanded curve and is rated $\psi = 2$—the very accuracy that earns `Qwen2.5-Coder-0.5B` (M_{12}, 0.9 GB) a $\psi = 4$, because the compact model clears the same bar with roughly 27× less footprint.

Model size alone does not dictate efficiency.: A small footprint is not inherently rewarded, nor is a large one inherently penalised on either axis; what matters is how efficiently a model converts the parameters it has. The smallest model, `codegen-350M-mono` (M_{11}, 0.74 GB), ranks among the weakest along every axis on both benchmarks—size–accuracy, size–energy, and the energy–accuracy ratings of RQ1—showing that a minimal footprint buys nothing once accuracy collapses. Conversely, `Qwen2.5-Coder-0.5B` (M_{12}, 0.92 GB) reaches the top Performance-Efficiency class on LiveCodeBench ($\psi = 5$) by extracting unusually high accuracy for its size, and `Seed-Coder-8B-Instruct` (M_{18}, 15.4 GB)

attains both top ratings there ($\psi = 5$, $\tau = 5$), converting a larger budget into genuinely high accuracy (0.32) while staying below the energy its size predicts. Because the size-adjustment charges footprint only where the accuracy trend saturates, Performance Efficiency is by design not orthogonal to size, though the association is controlled: it is uncorrelated with size on LiveCodeBench, whose trend rises steeply enough that the vertical deviation already carries the size signal (Spearman $\rho = 0.10$, $p = 0.57$), and mildly negative in the intended direction on CodeXGLUE, whose flat trend makes the size penalty the leading term ($\rho = -0.22$, $p = 0.23$). Structural Efficiency, derived from the non-saturating energy–size law, likewise shows no significant size correlation ($\rho = 0.04$ and 0.29; all $p > 0.05$). None of these associations is significant, so neither rating collapses into a size ranking. Consistent with this on LiveCodeBench, a majority (≈62%) of the below-median-size models (≤ 12.6 GB) fall below the size–accuracy demanded curve—under-performing the accuracy expected at their scale—and a similar majority sit above the size–energy curve, consuming more energy than their size warrants.

The geometry of the curves also encodes a complementary trade-off. Because the size–accuracy curve is increasing, two models above the demanded curve with identical accuracy are scored differently: the smaller one deviates further above the curve and earns the higher Performance Efficiency rating. Symmetrically, because the size–energy curve is increasing, two models, which both sit below the trend curve, with identical energy are separated in favor of the *larger* one, which sits further below its (higher) expected energy and earns the higher Structural Efficiency rating. Formally, for two models $m_1 = (A, E, S_1)$ and $m_2 = (A, E, S_2)$ with $S_1 < S_2$, m_1 wins on size–accuracy while m_2 wins on size–energy. In words, Performance Efficiency rewards reaching a given accuracy with fewer parameters, whereas Structural Efficiency penalizes a small model for spending as much energy as a larger one. Finally, deviation from the curves grows with scale, most visibly for energy: the largest models show the widest spread, consistent with a greater tendency to over-generate or to have been trained for longer contexts.

Joint behavior across the two efficiency axes.: Examining ψ and τ together shows that, on both benchmarks, a model is more likely to excel on one axis while lagging on the other than to succeed or fail on both. On LiveCodeBench, 4 models are strong on both ($\psi, \tau \geq 4$), 8 are weak on both ($\psi, \tau \leq 2$), and 15 are sharply split, scoring high on one axis (≥ 4) and low on the other (≤ 2), while the remaining 6 of the 33 fall in intermediate combinations. On CodeXGLUE, *no* model is simultaneously performance- and structurally efficient, 11 are weak on both, 8 are sharply split, and the remaining 14 fall in intermediate combinations. The complete absence of dual-efficient models on CodeXGLUE may reflect the difficulty of code summarization, namely producing an adequate docstring while keeping energy low for the model's size. The models weak on both axes (8 on LiveCodeBench, 11 on CodeXGLUE) under-deliver accuracy for their size while over-spending energy. For the larger members of this group this may indicate structural bloat, namely added parameters that neither improve accuracy nor energy efficiency, whereas for the smaller members (M_3 and M_{11} among them) it may reflect architectural or algorithmic inefficiency and a tendency to over-generate rather than any capability gained from scale.

Individual models illustrate the full range of trade-offs the two axes expose. `starcoderbase-1b` (M_2) is efficient *only* structurally, because with near-zero accuracy it expends very little energy, less than other models of its size, so it scores best on size–energy ($\tau = 5$ on LiveCodeBench and $\tau = 4$ on CodeXGLUE) but worst on size–accuracy ($\psi = 1$), having effectively traded capability for a small, low-energy profile. M_{29} is the mirror image on LiveCodeBench, because it achieves strong accuracy for a small 3.2 GB size ($\psi = 5$) but draws far more energy than its size warrants, sitting above the size–energy curve ($\tau = 2$), so the accuracy it buys is not worth its structural cost. Lastly, `Qwen2.5-Coder-0.5B` (M_{12}, 0.92 GB) shows why size context changes the verdict, because judged on energy and accuracy alone it is merely

moderate (rating 3 under both ***CIRC*** and ***OTER*** on LiveCodeBench) since its accuracy is low, yet relative to models of its size it is both more accurate and more energy-frugal, earning $\psi = 5$ and $\tau = 5$.

Robustness and generalization.: Several analyses confirm that the size-adjusted ratings are stable and that the underlying curves generalize. Leave-one-out analysis shows that removing any single model barely perturbs the rankings: mean rank drift is 0.028 (LiveCodeBench) and 0.035 (CodeXGLUE) for Performance Efficiency, and 0.026 and 0.034 for Structural Efficiency, with Kendall-$\tau \geq 0.98$ throughout. Under 5% uniform input perturbation, mean absolute rank drift stays below 0.14 for Performance Efficiency and 0.07 for Structural Efficiency. To probe whether the curves can extrapolate to unseen, larger models, we refit each curve on the smaller (denser) half of the cohort and predict the held-out larger (sparser) half. The error grows only modestly—accuracy RMSE rises from 0.068 to 0.086 (LiveCodeBench) and from 0.038 to 0.045 (CodeXGLUE), and energy RMSE from 4.80×10^5 to 5.69×10^5 J and from 3.84×10^5 to 4.98×10^5 J respectively—increases of 18–30% that are acceptable given the magnitude of the values and indicate that the curves remain predictive for footprints larger than those used to fit them. Overall, the size-adjusted ratings are stable to measurement noise and genuinely reward architectural density over brute-force capacity.

Summary

- Size alone does not dictate efficiency: the smallest model ranks among the weakest, while a compact 0.9 GB model and a dense 15.4 GB model both reach the top Performance-Efficiency class. Neither the Performance-Efficiency ($\rho = 0.10$ on LiveCodeBench, -0.22 on CodeXGLUE) nor the Structural-Efficiency ($\rho = 0.04, 0.29$) rating correlates significantly with size ($p > 0.05$).
- Performance Efficiency rewards reaching a given accuracy with fewer parameters, while Structural Efficiency rewards staying below the energy a model's size predicts—so for equal accuracy/energy the smaller model wins on size–accuracy and the larger on size–energy.
- Many models excel on one efficiency axis while lagging the other; no model is dual-efficient on CodeXGLUE, highlighting the difficulty of pairing accurate generation with structural restraint.
- The fitted curves are stable (leave-one-out and noise tests) and extrapolate to larger models with only modest error, supporting their use as fair, size-aware baselines.

6 Discussion

Characteristics of *CIRC* vs. *OTER*: Both methods solve the same MCDM problem but embody two different philosophies, and the choice between them depends on what a practitioner already knows about the objectives. ***CIRC*** fixes an absolute ideal at $(1, 1)$ and scores each model by its Euclidean distance to it. Because the reference is fixed rather than learned, ***CIRC*** assumes no correlation between energy and accuracy, lets a surplus in one objective compensate a deficit in the other without collapsing the score to zero, and—through its squared penalty—still requires balance to reach the top classes. Its pre-defined, count-independent regions make it deterministic and stable: a model's rating depends only on where it sits relative to the ideal, not on how many other models are present. This makes ***CIRC*** the safer default when the energy–accuracy relationship is unknown or when reproducibility across an evolving model set matters most.

OTER, by contrast, learns the empirical trade-off from the ecosystem and rewards models that beat the expectation at their efficiency level. It therefore credits a model that excels on a single objective when the prevailing trend makes that hard (demonstrating fairness) and penalizes models that are merely mediocre on both (demonstrating robustness). Its monotonically decreasing curve guarantees that a model dominating another on both axes always scores higher,

and its unbounded raw score v ($0 \leq v \leq \infty$) exposes each model's true magnitude, surfacing outliers and indicating where improvement is possible. In short, ***CIRC*** answers *"how far is this model from perfection?"* while ***OTER*** answers *"how does this model perform relative to what its peers achieve?"*—complementary lenses that, as RQ1 shows, agree on ordering and differ slightly in the middle classes.

Choosing among the three profiles and two methods: BRACE's decoupled design is deliberate: instead of a single opaque score, it offers three pairwise profiles that answer different deployment questions, and a practitioner can consult one, two, or all three. The energy–accuracy profile is the right lens when memory is not the binding constraint and the concern is sustainability at a given level of correctness. The size–energy profile matters when a deployment is memory-constrained and one needs to know whether a model wastes energy, or over-generates, relative to others of its footprint. The size–accuracy profile isolates whether a model earns its parameters in accuracy terms, flagging candidates for pruning or redesign. These two size-conditional view express that what separates models is not parameter count but how densely they convert that count into capability. Read together, the three profiles separate a model that is sustainable and accurate but oversized from one that is compact yet structurally wasteful—a distinction a monolithic score would hide. Within the energy–accuracy profile, ***CIRC*** suits users who want a fixed, reproducible mechanism, while ***OTER*** suits those who want models judged against the live frontier. When the objectives are not equally important, the weighted formulations (RQ3) let users tune priority continuously from accuracy-only to energy-only or vice versa while preserving monotonic, interpretable ratings, so an edge assistant and a cloud-based auditor can rely on the same framework under opposite priorities.

Inclusion of New Models: Because both methods are relative, expanding the model set can adjust ratings, but in a controlled and justified way (RQ2). ***CIRC*** is count-independent: as long as a newcomer's accuracy and energy fall within the observed range, every existing rating is unchanged, and a recomputation is required only when the newcomer extends an extreme. ***OTER*** additionally refits its expectation curve; we recommend refitting rather than freezing it, so that every model—old and new—is judged against the capability the field has since unlocked instead of an obsolete trend. In our expansion from 22 to 33 models the ordering was preserved (Kendall-τ = 1.00 for ***CIRC*** and $\geq$ 0.87 for ***OTER***), with the few ***OTER*** shifts limited to a single, upward class, indicating that both methods absorb new entrants reasonably.

Ground-Truth: Our problem is a multi-criteria decision-making (MCDM) whicb, by its nature, does not admit a single universally optimal solution. In Section 2, we first discuss the limitations of existing aggregation methods for this setting. We then develop two approaches based on established techniques: TOPSIS [43, 68], a widely used MCDM method that aggregates multiple criteria into a single score, and stochastic frontier analysis (SFA) [36, 44], an econometric framework for estimating optimal performance and inefficiency. We extend both methods with novel mechanisms tailored to our setting, enabling fair and robust five-class ratings of *CLM*. Finally, we evaluate the proposed methods through empirical analysis and dedicated robustness and fairness tests, demonstrating that their behavior remains consistent and logically aligned with the underlying energy–accuracy trade-offs.

Framework Extensibility and Task Agnosticism: While this study evaluates 33 models on code generation (text-to-code) and summarization (code-to-text) to systematically assess both logical reasoning and comprehension, BRACE is fundamentally task-agnostic. Our rating methodologies—both the MCDM trade-offs and scaling law derivations—operate strictly on the empirical distributions of observed metrics (accuracy, energy, footprint), making them completely independent of underlying task mechanics. By decoupling evaluation from implementation, users can test any *CLM* on any benchmark simply by updating configuration identifiers [10]. This enables practitioners to benchmark new

releases, evaluate domain-specific SE tasks, or seamlessly extend to arbitrary code benchmarks with minimal setup overhead and zero code modification. The framework also supports configurable rating ranges (*e.g.,* 1–10 instead of the conventional 1–5) to accommodate different granularity requirements. Because performance naturally fluctuates across tasks, evaluating models on an expanded suite yields a matrix of benchmark-specific ratings. To achieve a single global evaluation, practitioners can easily apply aggregation techniques to our three evaluation profiles (R^1, R^2, R^3) across these ratings to align with their precise operational priorities.

7 Implications

BRACE turns the abstract goal of sustainable yet capable AI into an actionable, interpretable rating that different stakeholders can act on directly. For software development teams and *GenAI practitioners*, the three profiles support concrete model selection: a team can pick the most accurate-and-efficient model when hardware is ample, switch to the size–energy profile when deploying on edge or consumer-grade GPUs, and use the weighted ratings to encode their own priority—accuracy-first for a mission-critical auditor, energy-first for an always-on coding assistant—without leaving the framework. Because the scale is a familiar 1–5 (akin to appliance energy labels but performance-aware), the choice stays legible to non-experts, while the unbounded raw scores remain available when finer separation within a class is needed.

For *model hosting platforms* such as HuggingFace, BRACE provides a reproducible badge that can sit beside a model card. ***CIRC***'s count-independent scoring means a newly uploaded model can be placed and rated without recomputing the entire leaderboard, while ***OTER*** can be refit periodically to keep the badge aligned with the evolving frontier. This lowers the barrier to surfacing sustainability information at the moment of selection, where it most influences adoption. *Companies* can likewise use these ratings as evidence of an environmentally conscious yet performance-driven posture, an increasingly visible market differentiator.

The size extension is what makes BRACE especially valuable to *model developers and researchers*. By scoring a model against the accuracy and energy expected for its footprint, the Performance- and Structural-Efficiency profiles act as a diagnostic: a model that fails to clear the accuracy its footprint demands, or exceeds the energy its footprint predicts, is flagged as token-starved, architecturally inefficient, oversized, or prone to over-generation. This directly counters the "Large Model Bias" that BRACE is designed to remove, rewarding architectural density over raw parameter count. Additionally, they can integrate the BRACE scoring mechanism into their CI/CD pipelines and enforce deployment policies based on predefined criteria. For example, a model may only be deployed if its rating exceeds a specified threshold or if it ranks within the top percentile of the other models. Researchers can further mine the three profiles to study which design choices lift a model above its peers and to locate gaps in the current ecosystem—for example, the scarcity of models that are jointly accurate and efficient, or the absence of dual-efficient models on summarization.

Finally, BRACE gives publishers, practitioners, and researchers a shared, transparent vocabulary for reasoning about where a model stands across the three dimensions that govern real-world deployment: correctness, energy, and size.

8 Related Works

General LLM Benchmarking: Numerous studies benchmark LLMs based on various criterias[97, 109]. For example, IFEVAL [110] evaluates models on instruction-following abilities using 25 verifiable tasks. Similarly, HELM [60] provides a holistic bench measuring accuracy, calibration, robustness, fairness, bias, toxicity, and efficiency for 30 models.

Sustainability Behavior Analysis: In parallel to evaluating LLMs on different benchmarks, studying the sustainability behavior of current LLMs and optimizing them have been widely investigated in recent literature [63, 72, 91]. Morrison *et al.* [74] holistically measured the resource exhausted throughout the development phase. Moreover, Google and Meta disclose their carbon footprint during their pretraining stages for the Gemma and Llama models, respectively [99, 101]. Other studies [83, 90] have examined the impact of optimization techniques on the energy consumption of AI pipeline stages, ranging from pre-training to inference phases.

LLM Sustainability and Benchmarking:

Several studies have focused on benchmarking LLMs on their environment impact. Argerich *et al.* [12] developed an energy profiler to measure energy consumption of an LLM during inference. Rajput *et al.* [84] benchmarked quantization techniques from an energy-efficiency perspective, providing insights for choosing sustainable models in resource-constrained settings. Samsi *et al.* [101] analyzed Llama variants (7B, 13B, 65B) across different hardware and configurations to highlight the benefits of power capping and improved GPU utilization. Ecologits [37] proposed a framework to estimate the environmental impact of proprietary models using regression-based energy and latency predictions. HuggingFace AI energy score benchmarked both proprietary and open-source models on ten tasks and reported 1-5 star ratings for their energy performance [64].

Joint Accuracy, Energy, and Size Benchmarking: As already discussed, benchmarking LLMs solely from a green-AI perspective overlooks functional correctness, making their adoption challenging since their true performance remains unexplored. Correctness alone, however, is not the only practical concern: a model's physical size governs whether it can be deployed under real hardware constraints, so a holistic comparison must weigh accuracy, energy, and scale together. We therefore first review works that jointly target accuracy and energy, and then the smaller body of work that additionally incorporates model size.

We observe a few works targeting both accuracy and energy aspects. Luccioni *et al.* [65] evaluated whether a model exhibits comparable performance regarding its emissions.

ZEUS [107] experimented with various language models and tasks and formulated a leaderboard with energy-accuracy related metrics. Other works study Pareto-optimal energy-accuracy trade-offs in specialized domains: Alizadeh *et al.* [7] focus on SE tasks, while Husom *et al.* [48] target embedded systems. Lastly, in the domain of scoring and ranking LLMs from a sustainability perspective, Pham *et al.* [81] scrutinize sustainability, correctness, and computation characteristics of Small Language Models (SLM). Kaplan *et al.* [55] proposed a novel metric, *i.e.,* delta carbon emission by computing the ratio of normalized carbon emission and f1-score. Jegham *et al.* [51] proposed a framework to predict sustainability data of proprietary models and used cross-efficiency DEA to rank models by performance relative to environmental cost.

A smaller body of work goes a step further and integrates model size alongside accuracy and energy. Gjorgjevikj *et al.* [39] rank models across accuracy, scale, and sustainability using the PROMETHEE II multi-criteria outranking method, letting users express trade-off preferences over the three dimensions. Fischer *et al.* [33] propose a unified framework that discretizes energy, accuracy, and size-related metrics independently and aggregates them into a single score through a weighted median.

Despite these advancements in the green AI field, a sound approach that jointly considers energy, accuracy, and model size while capturing their correlated behavior without sacrificing interpretability remains lacking. Moreover, the sustainability aspects of *CLM*s remain underexplored, limiting our ability to systematically compare their energy–accuracy–size trade-offs. Our framework addresses these gaps by providing seamless energy, accuracy, and size

computation on two widely used software engineering benchmarks. We introduced two tailored rating methods that classify models on a 1–5 scale based on a fair and robust assessment of the energy–accuracy trade-off, and complemented them with scaling-law-based performance and structural efficiency metrics that capture how effectively models convert their size into accuracy and energy savings. We then benchmarked all *CLM*s using these approaches to reveal the strengths and weaknesses of each model.

9 Threats to Validity

Internal Validity:

The main internal threat is that our energy readings could be distorted by measurement noise or background activity. To guard against this, we followed established energy-measurement practice [86]: we applied warm-up and cool-down phases, repeated every experiment three times, ran in-context CodeGreen mode for isolation, and cross-validated its readings against the *perf* CLI. We also measured energy only over the model-operation blocks, excluding setup overhead, and ran each model in a separate process so that GPU and CUDA cache residue from one run could not affect the next. A second internal threat is that ***OTER*** relies on algorithmic choices such as the polynomial degree, the outlier threshold, and the slope constraint. We tested 358 such configurations in RQ1 and found the ratings highly stable (mean Spearman $\rho = 0.982$), so no single choice drives our conclusions.

External Validity:

Our results may not generalize beyond the benchmarks, models, and hardware we used. First, we evaluate two SE tasks rather than many. We chose code generation and summarization because they cover the two opposite directions a CLM must handle—natural language to code and code to natural language—and therefore stress accuracy and energy in different ways; BRACE is also task-agnostic, so a new benchmark can be added through configuration alone (Section 5). Second, our cohort is limited to open-source models under 9B parameters, a bound imposed by our 32GB GPU, so larger or proprietary models could behave differently. To probe this, we showed that including new models on both ***CIRC*** and ***OTER*** did not distort and reordered ratings, in fact, even outliers are rewarded and penalized fairly. Secondly, we refit the size curves on the smaller half of the models and predicted the larger half (RQ4 5.4); the error grew only modestly, suggesting the framework would extend to larger footprints. Third, the scaling-law forms we adopt are not the only possible ones. However, we selected the forms that are established in the literature and address past limitations for both the size–accuracy and size–energy relationships. Finally, energy consumption depends on hardware. We repeated our measurements on a second machine and observed a consistent, proportional shift in energy with unchanged accuracy, so the relative ratings are expected to hold across platforms.

Construct Validity: The discrete 1–5 scale can place a boundary-adjacent model one class above or below a near-identical peer; this is inherent to any discretization, and we mitigate it by exposing the continuous raw scores (v for ***OTER*** and d for ***CIRC***), which we also prove preserve the underlying ordering, so that a single exceptional model may compress the visible distribution but never reorders it. For weighted ***OTER***, the piecewise score is non-smooth at the equal-weight seam; we designed it to remain fair and monotonic, and our continuity and sensitivity tests (RQ3 5.3) confirm that ratings change gradually rather than abruptly across the transition. Because ***OTER***'s curve is task-specific, the weight that best matches a user's intent can differ per benchmark, which our interpretable, visual weight sweep helps users identify.

A first construct threat is that both methods normalize energy and accuracy with min–max, which lets an extreme value reposition the others. We limit this through outlier removal and a large, consistently rated model set, and our leave-one-out and noise tests (RQ1 5.1, RQ4 5.4) confirm the ratings stay stable when individual models are removed

or inputs are perturbed by ±5%. A second threat is that the discrete 1–5 scale can place a model one class above or below a near-identical neighbor when it sits on a boundary. This is inherent to any discretization; we mitigate it by also reporting the continuous raw scores (v for ***OTER*** and d for ***CIRC***), which we prove preserve the underlying ordering. Consequently, an unusually strong or weak new model may compress the visible class distribution but cannot reverse the ranking. A final threat concerns the weighted ***OTER*** score, which is non-smooth at the equal-weight point. We designed this transition to remain fair and monotonic, and the continuity and sensitivity tests in RQ3 5.3 confirm that ratings change gradually rather than abruptly across it.

10 Threats to Validity

Internal Validity: The main internal threat is that our energy readings could be distorted by measurement noise or background activity. To guard against this, we followed established energy-measurement practice [86]: we applied warm-up and cool-down phases, repeated every experiment three times, ran CodeGreen in process mode to isolate the target process, and cross-validated its readings against the *perf* CLI. We also measured energy only over the model-operation blocks, excluding setup overhead, and ran each model in a separate process so that GPU and CUDA cache residue from one run could not affect the next. A second internal threat is that ***OTER*** relies on algorithmic choices—polynomial degree, outlier threshold, and slope constraint—and could, in principle, overfit a limited sample. The Least Expected Slope (LES) constraint and the Minimum Covariance Determinant outlier removal explicitly guard against overfitting, and our sweep of 358 configurations in RQ1 (Section 5.1) shows the ratings remain highly stable (mean Spearman $\rho = 0.982$), so no single design choice drives our conclusions.

External Validity: Our results may not generalize beyond the benchmarks, models, and hardware we used. First, we evaluate two SE tasks rather than many. We chose code generation and summarization because they cover the two opposite directions a *CLM* must handle—natural language to code and code to natural language—and therefore stress accuracy and energy in different ways; BRACE is also task-agnostic, so additional benchmarks can be added through configuration alone (Section 5), which would further refine the fitted ***OTER*** and scaling-law curves. Second, our cohort is limited to open-source models under 9B parameters, a bound imposed by our 32GB GPU, so larger or proprietary models could behave differently. To reduce this, we selected the most-downloaded models, so the cohort reflects the architectures practitioners most commonly use and spans from very small footprints to the largest our hardware admits. A related threat is that ***OTER***'s learned trend, as well as the size baselines, could shift if much larger models were introduced. We mitigate this in two ways: the recalibration analysis in RQ2 (Section 5.2) shows both methods absorb new entrants gracefully without reordering the originals, and the held-out test in RQ4 (Section 5.4)—refitting the size curves on the smaller half of the cohort and predicting the larger half—incurs only a modest error, suggesting the framework extends to larger footprints. Third, the scaling-law forms we adopt are not the only possible ones. We mitigate this by selecting forms that are established in the literature and address past limitations for both the size–accuracy and size–energy relationships. Finally, energy consumption depends on hardware. We repeated our measurements on a second machine and observed a consistent, proportional shift in energy with unchanged accuracy, so the relative ratings are expected to hold across platforms.

Construct Validity: A first threat is that the discrete 1–5 scale can place a boundary-adjacent model one class above or below a near-identical neighbor. This is inherent to any discretization; we mitigate it by also reporting the continuous raw scores (v for ***OTER*** and d for ***CIRC***), which we prove preserve the underlying ordering, so an unusually strong or weak model may compress the visible distribution but cannot reverse the ranking. Moreover, a model lying exactly on a boundary can reasonably belong to either adjacent class, so assigning it to one of them is not unfair. A second threat

concerns the weighted ***OTER*** score, which is non-smooth at the equal-weight seam. We designed this transition to remain fair and monotonic, and the continuity and sensitivity tests in RQ3 (Section 5.3) confirm that ratings change gradually rather than abruptly across it. Relatedly, because ***OTER***'s curve is task-specific, the weight that best matches a user's intent can differ per benchmark; our interpretable, visual weight sweep helps users identify the appropriate setting. A final threat concerns the interpretation of the size-efficiency ratings: a model lying above or below the size–accuracy or size–energy curve need not have a single, definite cause. A model below the accuracy curve may be token-starved, architecturally inefficient, or under-trained, while a model above the energy curve may be structurally bloated or simply over-generating. Rather than assert a unique explanation, we enumerate these plausible causes (Section 5.4) so that the rating flags *which* models deviate and points practitioners toward the likely factors to investigate.

11 Conclusions

We propose BRACE to benchmark *CLM*s in terms of functional correctness and energy efficiency. We further introduce two systematic rating methods, ***CIRC*** and ***OTER***, that score *CLM*s along unified accuracy-energy dimensions. Our experiments on widely used state-of-the-art code-to-text and text-to-code benchmarks show that ***CIRC*** provides a stable, deterministic ranking, whereas ***OTER*** is trend-aware, robust, and more sensitive to outliers. Overall, the resulting ratings act as portable energy–accuracy tags, helping software and AI practitioners identify models that achieve strong correctness at low energy cost.

We plan to extend this study by investigating the key factors that *strongly influence model accuracy*, offering valuable insights for future model developers to prioritize during their training stages. Furthermore, for ***OTER***, we desire to use more models to generate a universal task-specific function that can help providers target energy efficiency based on their accuracy, or vice versa. We also intend to broaden the cohort to additional architectures and settings—such as Mixture-of-Experts and larger models, varying inference configurations like batch size, and agentic workloads where energy accumulates across many calls. Lastly, we aim to provide an informative list of optimization techniques to obtain a better rating by investigating the effect of different factors, including hyperparameters and quantization.

References

[1] Mansour Zoubeirou a Mayaki and Victor Charpenay. 2025. Modeling Energy Consumption in Deep Learning Architectures Using Scaling Laws. (2025).

[2] Mohammad Abdollahi, Ruixin Zhang, Nima Shiri Harzevili, Jiho Shin, Song Wang, and Hadi Hemmati. 2025. Surveying the Benchmarking Landscape of Large Language Models in Code Intelligence. (2025).

[3] Saima Afrin, Joseph Call, Khai-Nguyen Nguyen, Oscar Chaparro, and Antonio Mastropaolo. 2025. Resource-Efficient & Effective Code Summarization. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE.

[4] Toufique Ahmed and Premkumar Devanbu. 2022. Few-shot training LLMs for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering*.

[5] Dennis Aigner, CA Knox Lovell, and Peter Schmidt. 1977. Formulation and estimation of stochastic frontier production function models. *Journal of econometrics* 6, 1 (1977).

[6] Ibrahim M Alabdulmohsin, Behnam Neyshabur, and Xiaohua Zhai. 2022. Revisiting neural scaling laws in language and vision. *Advances in Neural Information Processing Systems* 35 (2022), 22300–22312.

[7] Negar Alizadeh, Boris Belchev, Nishant Saurabh, Patricia Kelbert, and Fernando Castor. 2025. Language Models in Software Development Tasks: An Experimental Analysis of Energy and Accuracy. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE.

[8] Abdullah Mahmoud Almasri and Luis Borges Gouveia. 2021. Star-rating evaluation model for rating the energy-efficiency level of android google play apps. *International Journal of Electrical and Computer Engineering* 11, 2 (2021).

[9] Tiago L Alves, Christiaan Ypma, and Joost Visser. 2010. Deriving metric thresholds from benchmark data. In *2010 IEEE international conference on software maintenance*. IEEE, 1–10.

[10] Anon. 2025. GitHub - tmp351/BRACE: Unified Benchmarking Accuracy and Energy of Code Language Models — github.com. https://github.com/tmp351/BRACE. [Accessed 13-10-2025].

[11] Anthropic. 2025. GitHub - anthropics/prompt-eng-interactive-tutorial: Anthropic's Interactive Prompt Engineering Tutorial — github.com. https://github.com/anthropics/prompt-eng-interactive-tutorial. [Accessed 16-10-2025].

[12] Mauricio Fadel Argerich and Marta Patiño-Martínez. 2024. Measuring and improving the energy efficiency of large language models inference. *IEEE Access* 12 (2024).

[13] Arne Berger, Matthias Knauer, and Christof Büskens. 2018. Pareto Front Interpolation Based on Parametric Sensitivity Analysis in a Bi-Objective Setting. *PAMM* 18, 1 (2018).

[14] Nuran Bradley. 2007. *The response surface methodology.* Ph. D. Dissertation. Indiana University South Bend.

[15] Jonathan Bright, Florence Enock, Saba Esnaashari, John Francis, Youmna Hashem, and Deborah Morgan. 2025. Generative AI is already widespread in the public sector: evidence from a survey of UK public sector professionals. *Digit. Gov.: Res. Pract.* 6, 1 (2025). doi:10.1145/3700140

[16] Alexander EI Brownlee, Jason Adair, Saemundur O Haraldsson, and John Jabbo. 2021. Exploring the accuracy–energy trade-off in machine learning. In *2021 IEEE/ACM International Workshop on Genetic Improvement (GI).* IEEE.

[17] David Budescu. 1980. A Note On Polynomial Regression. *Multivariate Behavioral Research* 15 (1980). doi:10.1207/s15327906mbr1504_7

[18] Ethan Caballero, Kshitij Gupta, Irina Rish, and David Krueger. 2023. Broken Neural Scaling Laws. In *The Eleventh International Conference on Learning Representations.* https://openreview.net/forum?id=sckjveqlCZ

[19] Francisco Caravaca, Ángel Cuevas, and Rubén Cuevas. 2025. From Prompts to Power: Measuring the Energy Footprint of LLM Inference. *arXiv preprint arXiv:2511.05597* (2025).

[20] Catherine Cazals, Jean-Pierre Florens, and Léopold Simar. 2002. Nonparametric frontier estimation: a robust approach. *Journal of econometrics* 106, 1 (2002).

[21] Aydın Çelen. 2014. Comparative analysis of normalization procedures in TOPSIS method: with an application to Turkish deposit banking market. *Informatica* (2014).

[22] A. Charnes, W.W. Cooper, and E. Rhodes. 1978. Measuring the efficiency of decision making units. *European Journal of Operational Research* 2, 6 (1978). doi:10.1016/0377-2217(78)90138-8

[23] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[24] David Christensen and Kirk Payne. 1992. Cost performance index stability: fact or fiction? *Journal of Parametrics* (1992).

[25] Jae-Won Chung, Jiachen Liu, Jeff J. Ma, Ruofan Wu, Oh Jun Kweon, Yuxuan Xia, Zhiyu Wu, and Mosharaf Chowdhury. 2025. The ML.ENERGY Benchmark: Toward Automated Inference Energy Measurement and Optimization. *arXiv preprint arXiv:2505.06371* (2025).

[26] Charles W Cobb and Paul H Douglas. 1928. A theory of production. *The American economic review* (1928).

[27] R. De Maesschalck, D. Jouan-Rimbaud, and D.L. Massart. 2000. The Mahalanobis distance. *Chemometrics and Intelligent Laboratory Systems* 50, 1 (2000). doi:10.1016/S0169-7439(99)00047-7

[28] Zhengxiao Du, Aohan Zeng, Yuxiao Dong, and Jie Tang. 2024. Understanding emergent abilities of language models from the loss perspective. *Advances in neural information processing systems* 37 (2024), 53138–53167.

[29] Hugging Face. 2025. Hugging Face. https://huggingface.co/. Accessed: 2025-10-10.

[30] Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Chukwunyere Osi, Prateek Sharma, Fan Chen, and Lei Jiang. 2024. LLMCarbon: Modeling the End-to-End Carbon Footprint of Large Language Models. In *The Twelfth International Conference on Learning Representations.*

[31] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE).* doi:10.1109/ICSE-FoSE59343.2023.00008

[32] Emile Dos Santos Ferreira, Andrei Paleyes, and Neil D Lawrence. 2026. Optimising for Energy Efficiency and Performance in Machine Learning. In *2026 IEEE/ACM 5th International Conference on AI Engineering–Software Engineering for AI (CAIN).* IEEE.

[33] Raphael Fischer, Matthias Jakobs, Sascha Mücke, and Katharina Morik. 2022. A unified framework for assessing energy efficiency of machine learning. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases.* Springer.

[34] Clémentine Fourrier, Nathan Habib, Alina Lozovskaya, Konrad Szafer, and Thomas Wolf. 2024. Open LLM Leaderboard v2.

[35] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2024. The Language Model Evaluation Harness. doi:10.5281/zenodo.12608602

[36] Chunmian Ge and Ke-Wei Huang. 2014. Analyzing the economies of scale of software as a service software firms: A stochastic frontier approach. *IEEE Transactions on Engineering Management* 61, 4 (2014).

[37] GenAI Impact. 2025. EcoLogits Documentation. https://ecologits.ai/latest/. Accessed 2025-09-09.

[38] GitHub. 2025. *GitHub Copilot.* GitHub, Inc. https://github.com/features/copilot

[39] Ana Gjorgjevikj, Ana Nikolikj, Barbara Koroušić Seljak, and Tome Eftimov. 2025. User-defined trade-offs in LLM benchmarking: balancing accuracy, scale, and sustainability. *Knowledge-Based Systems* (2025).

[40] Yash Goel, Ayan Sengupta, and Tanmoy Chakraborty. 2025. Position: Enough of Scaling LLMs! Lets Focus on Downscaling. *arXiv preprint arXiv:2505.00985* (2025).

[41] María D Guillen, Juan Aparicio, José L Zofío, and Victor J Espana. 2024. Improving the predictive accuracy of production frontier models for efficiency measurement using machine learning: The LSB-MAFS method. *Computers & Operations Research* 171 (2024).

[42] Janis Hardwick and Quentin F Stout. 2014. Optimal reduced isotonic regression. *arXiv preprint arXiv:1412.2844* (2014).

[43] S Haseena, M Blessa Binolin Pepsi, and S Saroja. 2020. Multi criteria decision making technique for machine learning algorithms: iterative and noniterative algorithms. *International Journal Of Scientific & Technology Research* 9, 1 (2020), 2392–2403.

[44] Riko Hendrawan, Kristian Nugroho, and Gayuh Permana. 2020. Efficiency Perspective on Telecom Mobile Data Traffic. *GATR Journal of Business and Economics Review* 5 (03 2020), 38–44. doi:10.35609/jber.2020.5.1(5)

[45] Danny Hernandez, Jared Kaplan, Tom Henighan, and Sam McCandlish. 2021. Scaling laws for transfer. *arXiv preprint arXiv:2102.01293* (2021).

[46] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8 (2024). doi:10.1145/3695988

[47] Mia Hubert and Michiel Debruyne. 2010. Minimum covariance determinant. *Wiley interdisciplinary reviews: Computational statistics* 2, 1 (2010).

[48] Erik Johannes Husom, Arda Goknil, Merve Astekin, Lwin Khin Shar, Andre Kåsen, Sagar Sen, Benedikt Andreas Mithassel, and Ahmet Soylu. 2025. Sustainable LLM inference for edge AI: Evaluating quantized llms for energy efficiency, output accuracy, and inference latency. *ACM Transactions on Internet of Things* (2025).

[49] Ching-Lai Hwang and Kwangsun Yoon. 1981. *Methods for Multiple Attribute Decision Making*. Springer Berlin Heidelberg. doi:10.1007/978-3-642-48318-9_3

[50] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations*.

[51] Nidhal Jegham, Marwan Abdelatti, Lassad Elmoubarki, and Abdeltawab Hendawi. 2025. How hungry is AI? benchmarking energy, water, and carbon footprint of LLM inference. *arXiv preprint arXiv:2505.09598* (2025).

[52] Hamed Jelodar, Mohammad Meymani, and Roozbeh Razavi-Far. 2025. Large language models (LLMs) for source code analysis: applications, models and datasets. *arXiv preprint arXiv:2503.17502* (2025).

[53] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2025. A Survey on Large Language Models for Code Generation. *ACM Trans. Softw. Eng. Methodol.* (2025). doi:10.1145/3747588

[54] Yunho Jin, Gu-Yeon Wei, and David Brooks. 2025. The Energy Cost of Reasoning: Analyzing Energy Usage in LLMs with Test-time Compute. *arXiv preprint arXiv:2505.14733* (2025).

[55] Angelika Kaplan, Jan Keim, Lukas Greiner, Ralf Sieger, Raffaela Mirandola, and Ralf Reussner. 2025. Responsible and Sustainable AI: Considering Energy Consumption in Automated Text Classification Evaluation Tasks. In *Proceedings of the 2025 IEEE/ACM 9th International Workshop on Green and Sustainable Software (GREENS '25)*. IEEE Press. doi:10.1109/GREENS66463.2025.00016

[56] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).

[57] Jakub Krajewski, Amitis Shidani, Dan Busbridge, Sam Wiseman, and Jason Ramapuram. 2026. Revisiting the Scaling Properties of Downstream Metrics in Large Language Model Training. *The Fourteenth International Conference on Learning Representations* (2026). https://openreview.net/forum?id=YnJ2s4WeNF

[58] Arindam Kundu, Sumit Kumar, Nutan Kumar Tomar, and Shiv Kumar Gupta. 2016. Call option price function in Bernstein polynomial basis with no-arbitrage inequality constraints. *Journal of Inequalities and Applications* 2016, 1 (2016).

[59] Pengfei Li, Jianyi Yang, Mohammad A Islam, and Shaolei Ren. 2025. Making AI less' thirsty'. *Commun. ACM* 68, 7 (2025).

[60] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. 2022. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110* (2022).

[61] Chin-Yew Lin and Franz Josef Och. 2004. Orange: a method for evaluating automatic evaluation metrics for machine translation. In *COLING 2004: Proceedings of the 20th International Conference on Computational Linguistics*.

[62] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, MING GONG, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie LIU. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*.

[63] Alexandra Sasha Luccioni, Sylvain Viguier, and Anne-Laure Ligozat. 2023. Estimating the carbon footprint of bloom, a 176b parameter language model. *Journal of machine learning research* 24 (2023).

[64] Sasha Luccioni, Boris Gamazaychikov, Emma Strubell, Sara Hooker, Yacine Jernite, Carole-Jean Wu, and Margaret Mitchell. 2025. AI Energy Score Leaderboard - February 2025.

[65] Sasha Luccioni, Yacine Jernite, and Emma Strubell. 2024. Power hungry processing: Watts driving the cost of AI deployment?. In *Proceedings of the 2024 ACM conference on fairness, accountability, and transparency*.

[66] B Mahaboob, KA Ajmath, B Venkateswarlu, C Narayana, and J Peter Praveen. 2019. On Cobb-Douglas production function model. In *AIP Conference Proceedings*. AIP Publishing LLC.

[67] Felipe Maia Polo, Seamus Somerstep, Leshem Choshen, Yuekai Sun, and Mikhail Yurochkin. 2026. Sloth: scaling laws for llm skills to predict multi-benchmark performance across families. *Advances in Neural Information Processing Systems* (2026).

[68] Miguel Angel Quiroz Martinez, Eduardo Xavier Moreira Bermello, Wilmer Paul Carrillo Leon, and Luis Briones. 2022. A Hybrid Approach Entropy-TOPSIS for the Selection of Machine Learning Classifiers for Software Defect Prediction. *Intelligent Human Systems Integration (IHSI*

2022): Integrating People and Intelligent Systems 22, 22 (2022).

[69] Elio Masciari, Vincenzo Moscato, Enea Vincenzo Napolitano, Gian Marco Orlando, Marco Perillo, and Diego Russo. 2026. Are LLMs Ready for TOON? Benchmarking Structural Correctness-Sustainability Trade-offs in Novel Structured Output Formats. *arXiv preprint arXiv:2601.12014* (2026).

[70] Antonio Mastropaolo, Camilo Escobar-Velásquez, and Mario Linares-Vásquez. 2025. From triumph to uncertainty: The journey of software engineering in the AI era. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025).

[71] Manoj Mathew, Sagar Sahu, and Ashish Kumar Upadhyay. 2017. Effect of normalization techniques in robot selection using weighted aggregated sum product assessment. *Int. J. Innov. Res. Adv. Stud* (2017).

[72] Joseph McDonald, Baolin Li, Nathan Frey, Devesh Tiwari, Vijay Gadepally, and Siddharth Samsi. 2022. Great Power, Great Responsibility: Recommendations for Reducing Energy for Training Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2022*. Association for Computational Linguistics. doi:10.18653/v1/2022.findings-naacl.151

[73] Ali Mohaghar. 2013. Developing TOPSIS method using statistical normalization for selecting knowledge management strategies. *Journal of industrial engineering and management* (2013).

[74] J Morrison, C Na, J Fernandez, T Dettmers, E Strubell, and J Dodge. 2025. Holistically Evaluating the Environmental Impact of Creating Language Models. arXiv 2025. *arXiv preprint arXiv:2503.05804* (2025).

[75] Sophia Nguyen, Beihao Zhou, Yi Ding, and Sihang Liu. 2024. Towards sustainable large language model serving. *ACM SIGENERGY Energy Informatics Review* 4, 5 (2024).

[76] Chenxu Niu, Wei Zhang, Yongjian Zhao, and Yong Chen. 2025. Energy efficient or exhaustive? benchmarking power consumption of llm inference engines. *ACM SIGENERGY Energy Informatics Review* (2025).

[77] Michael Nordine, Anton Schwarz, Renana Bruckstein, Hanns-Christian Gunga, and Oliver Opatz. 2022. The human dive reflex during consecutive apnoeas in dry and immersive environments: magnitude and synchronicity. *Frontiers in Physiology* 12 (2022), 725361.

[78] Omer Onalan and Hulya Basegmez. 2018. Estimation of economic growth using Grey Cobb-Douglas production function: An application for US economy. *Journal of Business Economics and Finance* (2018).

[79] Zuzanna Osika, Jazmin Zatarain Salazar, Diederik M. Roijers, Frans A. Oliehoek, and Pradeep K. Murukannaiah. 2023. What lies beyond the pareto front? a survey on decision-support methods for multi-objective optimization. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI '23)*. doi:10.24963/ijcai.2023/755

[80] David Owen. 2024. How predictable is language model benchmark performance? *arXiv preprint arXiv:2401.04757* (2024).

[81] Nghiem Thanh Pham, Tung Kieu, Duc-Manh Nguyen, Son Ha Xuan, Nghia Duong-Trung, and Danh Le-Phuoc. 2025. SLM-Bench: A Comprehensive Benchmark of Small Language Models on Environmental Impacts–Extended Version. *arXiv preprint arXiv:2508.15478* (2025).

[82] Ketai Qiu, Niccolò Puccinelli, Matteo Ciniselli, and Luca Di Grazia. 2025. From Today's Code to Tomorrow's Symphony: The AI Transformation of Developer's Routine by 2030. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025). doi:10.1145/3709353

[83] Saurabhsingh Rajput, Mootez Saad, and Tushar Sharma. 2026. Tu(r)ning AI Green: Exploring Energy Efficiency Cascading With Orthogonal Optimizations. *IEEE Software* 43, 2 (2026), 32–41. doi:10.1109/MS.2025.3645090

[84] Saurabhsingh Rajput and Tushar Sharma. 2024. Benchmarking Emerging Deep Learning Quantization Methods for Energy Efficiency. In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*. doi:10.1109/ICSA-C63560.2024.00049

[85] Saurabhsingh Rajput and Tushar Sharma. 2026. CodeGreen: Towards Improving Precision and Portability in Software Energy Measurement. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering* (Concordia University, Montreal, QC, Canada) *(FSE Companion '26)*. Association for Computing Machinery, New York, NY, USA, 247–251. doi:10.1145/3803437.3806429

[86] Saurabhsingh Rajput, Tim Widmayer, Ziyuan Shang, Maria Kechagia, Federica Sarro, and Tushar Sharma. 2024. Enhancing energy-awareness in deep learning through fine-grained energy measurement. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024).

[87] Peter Rousseeuw. 1985. Multivariate estimation with high breakdown point. *Mathematical statistics and applications* 8 (1985).

[88] Peter J Rousseeuw. 1984. Least median of squares regression. *Journal of the American statistical association* 79 (1984).

[89] Yangjun Ruan, Chris J Maddison, and Tatsunori Hashimoto. 2024. Observational scaling laws and the predictability of langauge model performance. *Advances in Neural Information Processing Systems* (2024).

[90] Mootez Saad, José Antonio Hernández López, Boqi Chen, Dániel Varró, and Tushar Sharma. 2025. An Adaptive Language-Agnostic Pruning Method for Greener Language Models for Code. *Proceedings of the ACM on Software Engineering* 2, FSE (2025).

[91] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, and Vijay Gadepally. 2023. From words to watts: Benchmarking the energy costs of large language model inference. In *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE.

[92] Scikit-Learn. 2025. Isotonic Regression. https://scikit-learn.org/stable/modules/isotonic.html. Accessed 2025-09-24.

[93] Won-Gyo Seo and Se-Jin Han. 2017. Comparison of the effects on the pharyngeal airway space of maxillary protraction appliances according to the methods of anchorage. *Maxillofacial plastic and reconstructive surgery* 39, 1 (2017), 3.

[94] Zhigang Shen, Ashkan Hassani, and Qian Shi. 2016. Multi-objective time-cost optimization using Cobb-Douglas production function and hybrid genetic algorithm. *Journal of Civil Engineering and Management* (2016).

[95] Olivier Sobrie, Nicolas Gillis, Vincent Mousseau, and Marc Pirlot. 2018. UTA-poly and UTA-splines: additive value functions with polynomial marginals. *European Journal of Operational Research* 264, 2 (2018).

[96] Ankita Nandkishor Sontakke, Manasi Patwardhan, Lovekesh Vig, Raveendra Kumar Medicherla, Ravindra Naik, and Gautam Shroff. 2022. Code summarization: Do transformers really understand code?. In *Deep Learning for Code Workshop*.

[97] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adri Garriga-Alonso, et al. 2023. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on machine learning research* (2023).

[98] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2020. Energy and policy considerations for modern deep learning research. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34.

[99] Gemma Team, T Mesnard, C Hardin, R Dadashi, S Bhupatiraju, S Pathak, L Sifre, M Rivière, MS Kale, J Love, et al. [n. d.]. Gemma: Open models based on gemini research and technology. arXiv 2024. *arXiv preprint arXiv:2403.08295* ([n. d.]).

[100] Kaoru Tone. 2001. A slacks-based measure of efficiency in data envelopment analysis. *European journal of operational research* 130, 3 (2001).

[101] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[102] Gwo-Hshiung Tzeng and Jih-Jeng Huang. 2011. *Multiple attribute decision making: methods and applications*. CRC press.

[103] Ruiqi Wang, Jiyu Guo, Cuiyun Gao, Guodong Fan, Chun Yong Chong, and Xin Xia. 2025. Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering. *Proc. ACM Softw. Eng.* 2, ISSTA (2025). doi:10.1145/3728963

[104] Jinfeng Wen, Zhenpeng Chen, Jianshu Zhao, Federica Sarro, Haodi Ping, Ying Zhang, Shangguang Wang, and Xuanzhe Liu. 2025. SCOPE: Performance testing for serverless computing. *ACM Transactions on Software Engineering and Methodology* (2025).

[105] Chaojun Xiao, Jie Cai, Weilin Zhao, Biyuan Lin, Guoyang Zeng, Jie Zhou, Zhi Zheng, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. Densing law of llms. *Nature Machine Intelligence* (2025).

[106] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[107] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. 2023. Zeus: Understanding and Optimizing GPU Energy Consumption of DNN Training. In *USENIX NSDI*.

[108] Matt Young. 2006. The stone-weierstrass theorem. In *MATH 328 Notes*. Queen's University at Kingston.

[109] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging LLM-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023).

[110] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-Following Evaluation for Large Language Models. arXiv:2311.07911 [cs.CL]

[111] Stanley Zionts. 1979. MCDM—If not a roman numeral, then what? *Interfaces* 9, 4 (1979).

[112] Johannes Zschache and Tilman Hartwig. 2025. Comparing energy consumption and accuracy in text classification inference. *arXiv preprint arXiv:2508.14170* (2025).

[113] Fida Zubair, Maryam Al-Hitmi, and Cagatay Catal. 2025. The use of large language models for program repair. *Computer Standards & Interfaces* 93 (2025). doi:10.1016/j.csi.2024.103951